

Neural Network, Embeddings and Word2Vec

Academic Year 2025/2026

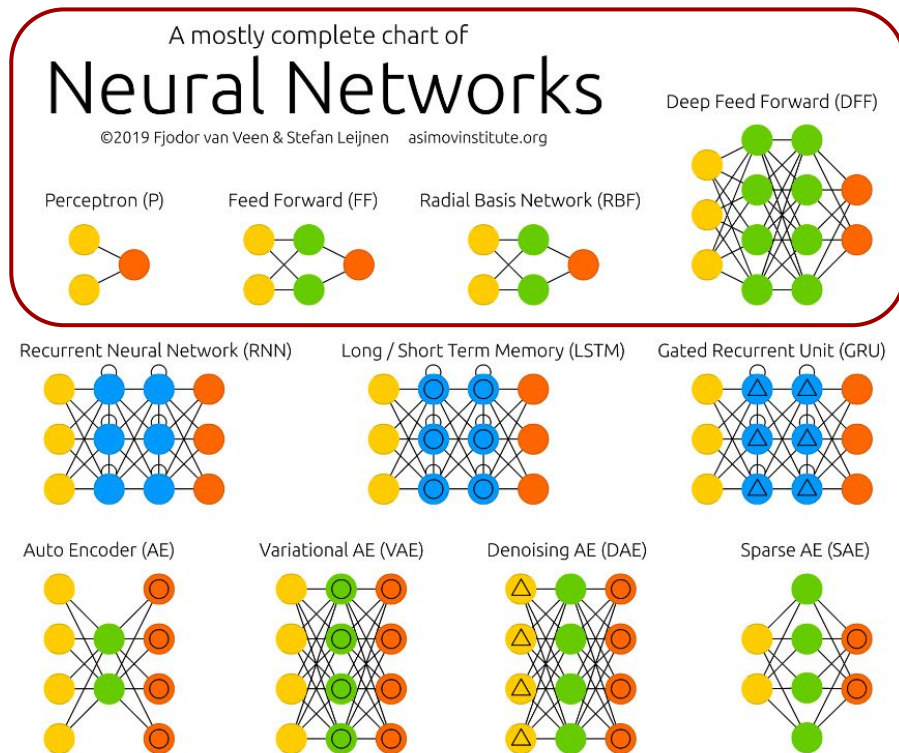
Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
**Master Degree in
Computer Engineering**

Neural Network

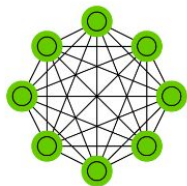
Neural Network Topology Overview

- Input Cell
- Backfed Input Cell
- △ Noisy Input Cell
- Hidden Cell
- Probabilistic Hidden Cell
- △ Spiking Hidden Cell
- Capsule Cell
- Output Cell
- Match Input Output Cell
- Recurrent Cell
- Memory Cell
- △ Gated Memory Cell
- Kernel
- Convolution or Pool

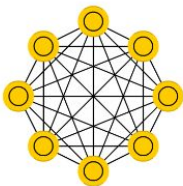


Neural Network Topology Overview

Markov Chain (MC)



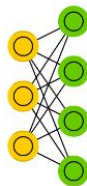
Hopfield Network (HN)



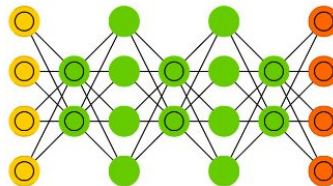
Boltzmann Machine (BM)



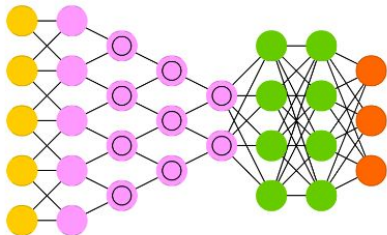
Restricted BM (RBM)



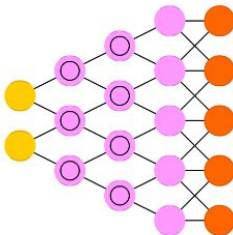
Deep Belief Network (DBN)



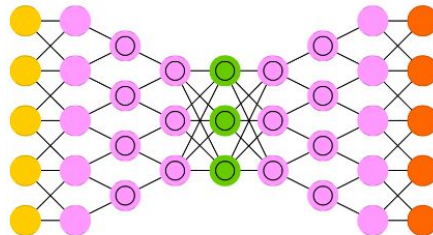
Deep Convolutional Network (DCN)



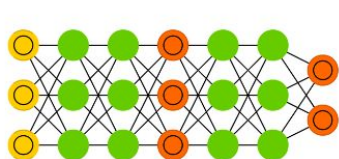
Deconvolutional Network (DN)



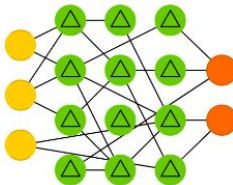
Deep Convolutional Inverse Graphics Network (DCIGN)



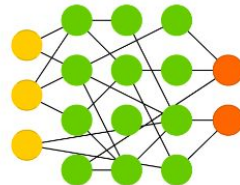
Generative Adversarial Network (GAN)



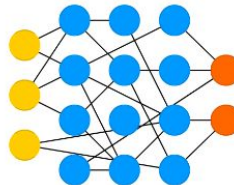
Liquid State Machine (LSM)



Extreme Learning Machine (ELM)

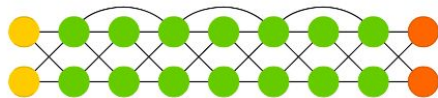


Echo State Network (ESN)

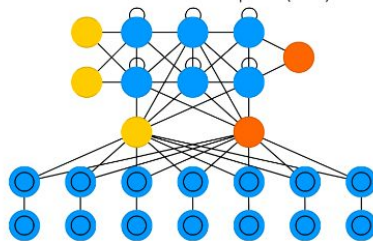


Neural Network Topology Overview

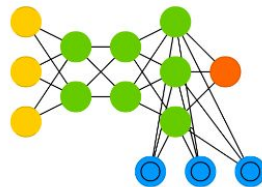
Deep Residual Network (DRN)



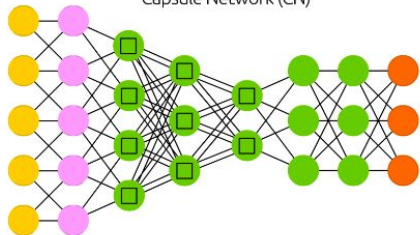
Differentiable Neural Computer (DNC)



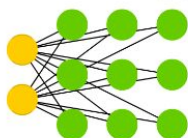
Neural Turing Machine (NTM)



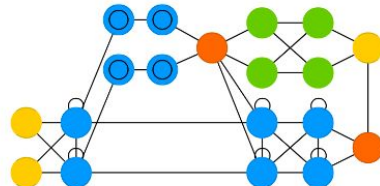
Capsule Network (CN)



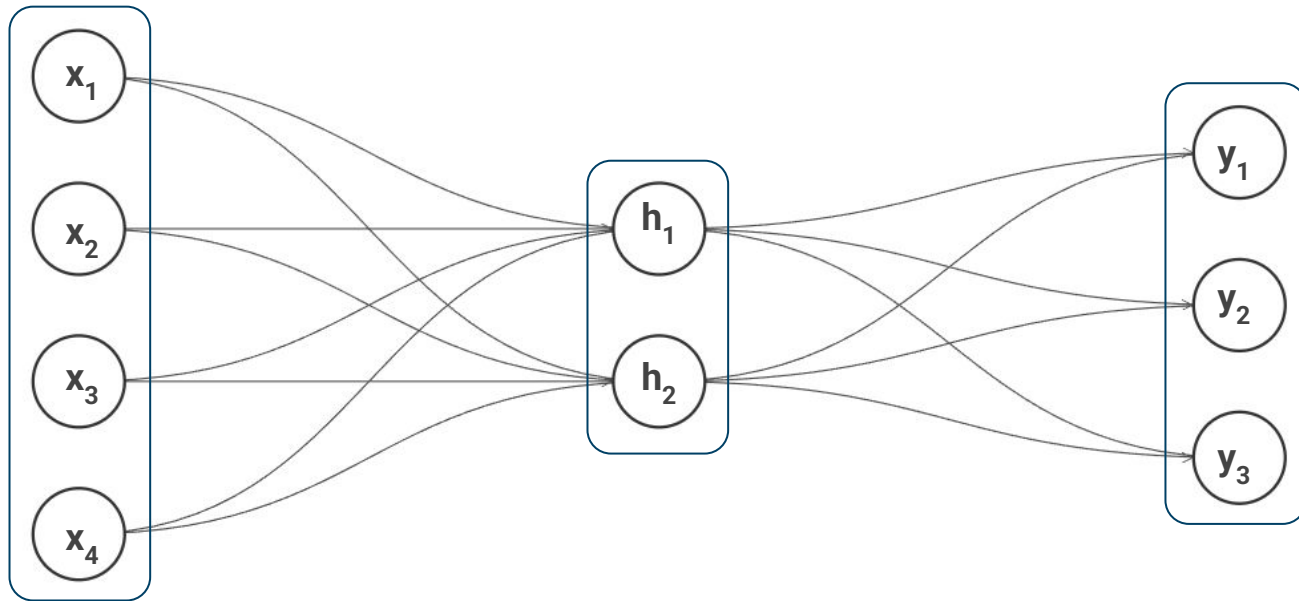
Kohonen Network (KN)



Attention Network (AN)



The structure of a simple Neural Network

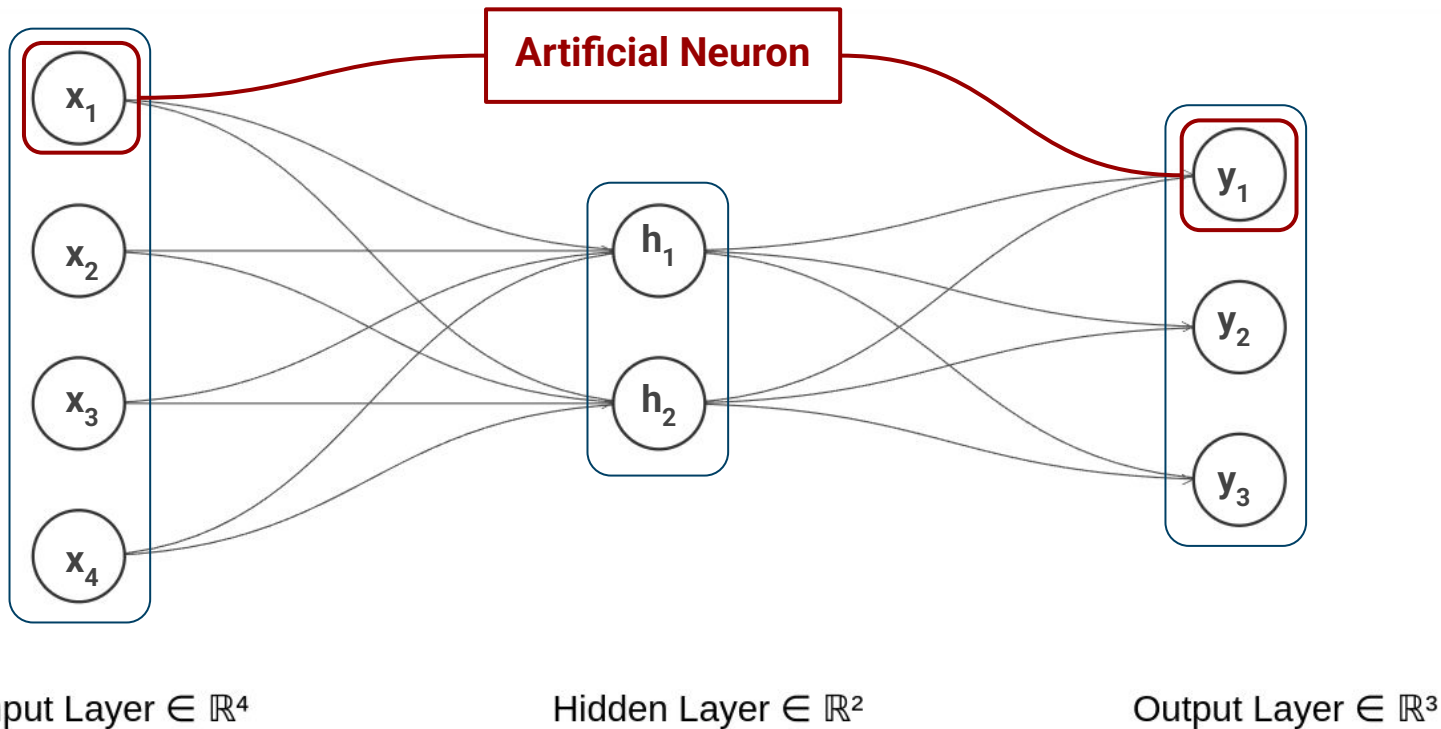


Input Layer $\in \mathbb{R}^4$

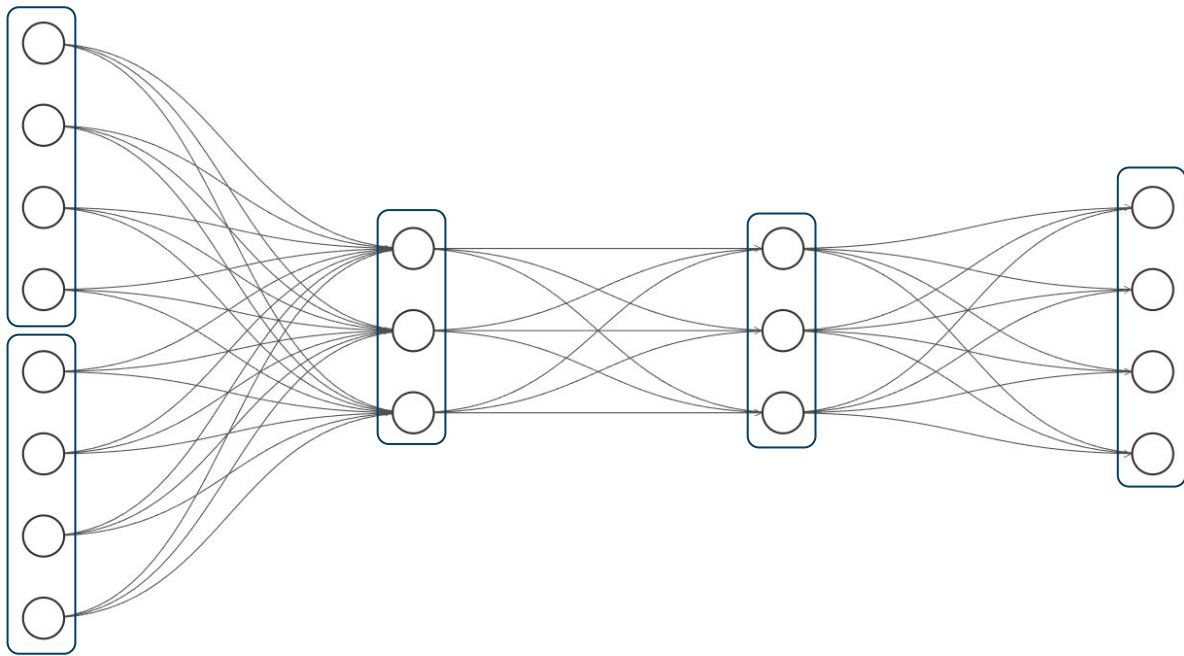
Hidden Layer $\in \mathbb{R}^2$

Output Layer $\in \mathbb{R}^3$

The structure of a simple Neural Network



A more complex Neural Network

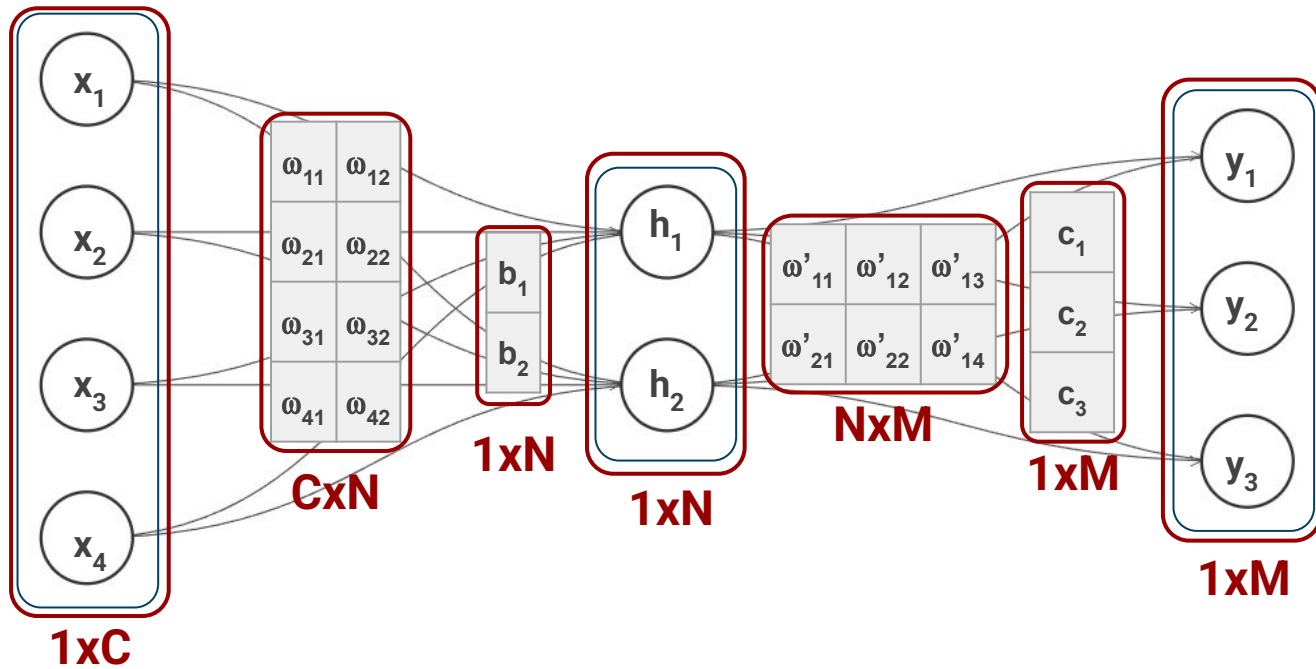


How many hidden layers?

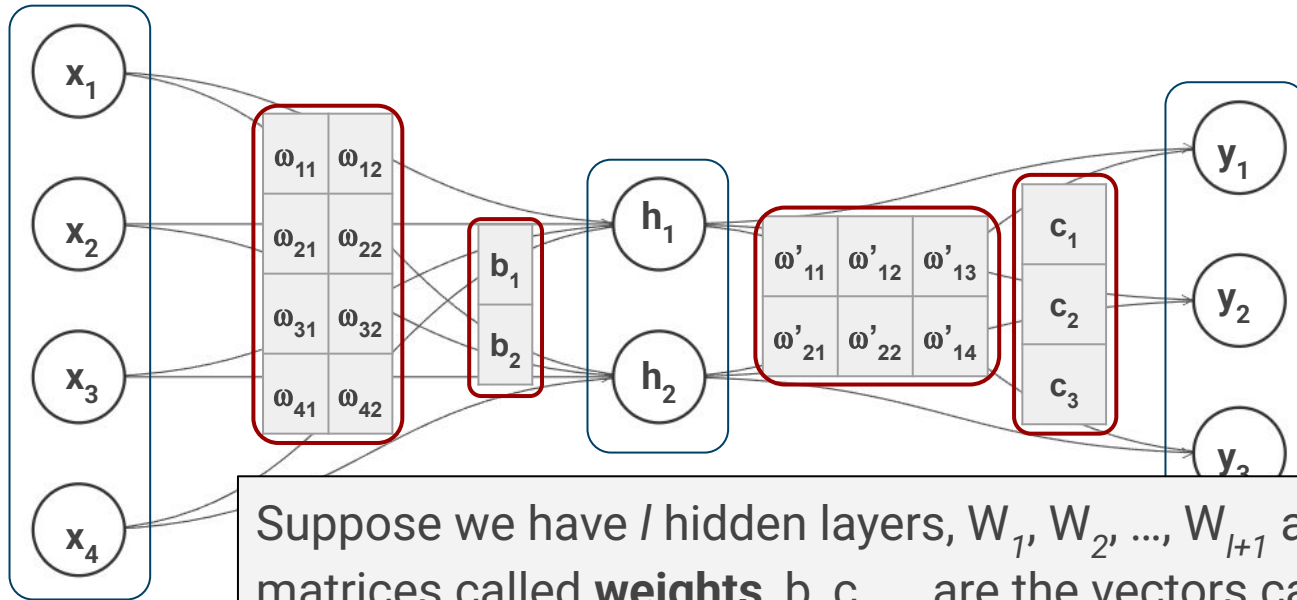
How many nodes in each hidden layers?

How to interconnect nodes from one layer to the next?

The structure of a simple Neural Network

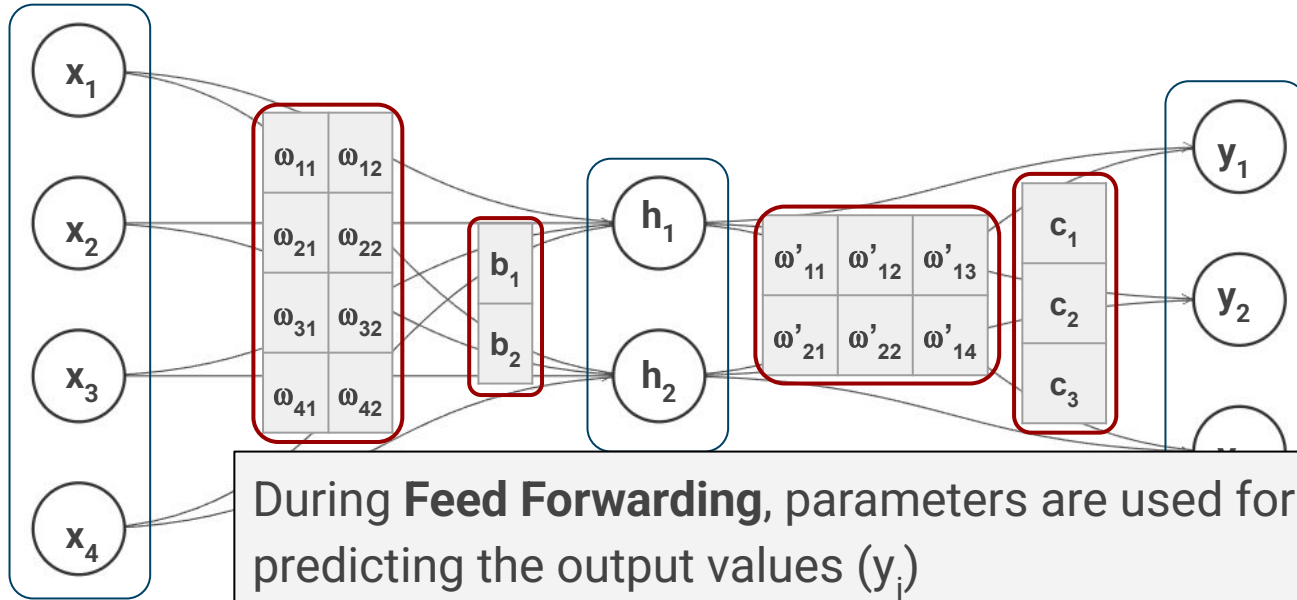


The structure of a simple Neural Network

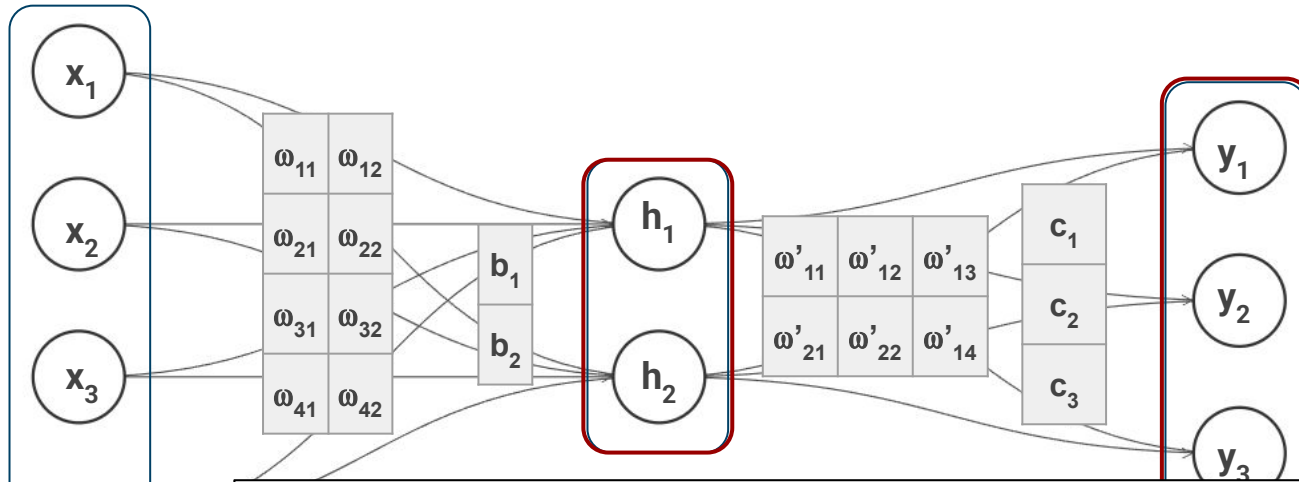


Suppose we have l hidden layers, $W_1, W_2, \dots, W_{l+1}$ are the matrices called **weights**, $b, c, \dots$, are the vectors called **bias**. These are **parameters**. Our objective is to learn the **best values** for these parameters.

Neural Network: two phases



Neural Network: activation functions



The values in the hidden layer and in the output layer are usually computed applying an **activation function**, that calculates the output of the node based on its individual inputs and their weights

Typical Activation functions for classification

Name	Function	Derivative
Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$	$\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$
Softmax	$\mu(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$	$\frac{d\mu(x_i)}{dx_j} = \begin{cases} \mu(x_i)(1 - \mu(x_i)) & \text{if } i = j \\ -\mu(x_i)\mu(x_j) & \text{if } i \neq j \end{cases}$

Considerations on Sigmoid

1. The function is continuous
2. The function is differentiable
3. For large positive x , the function approaches 1
4. For large negative x , the function approaches 0
5. So it ranges between 0 and 1
6. The output can be interpreted as a probability

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

x	$\sigma(x)$
10	0,99995
8	0,99966
-2	0,11920
4	0,98201
-3	0,04743
9	0,99988
2	0,88080
1	0,73106
-5	0,00669

Considerations on Softmax

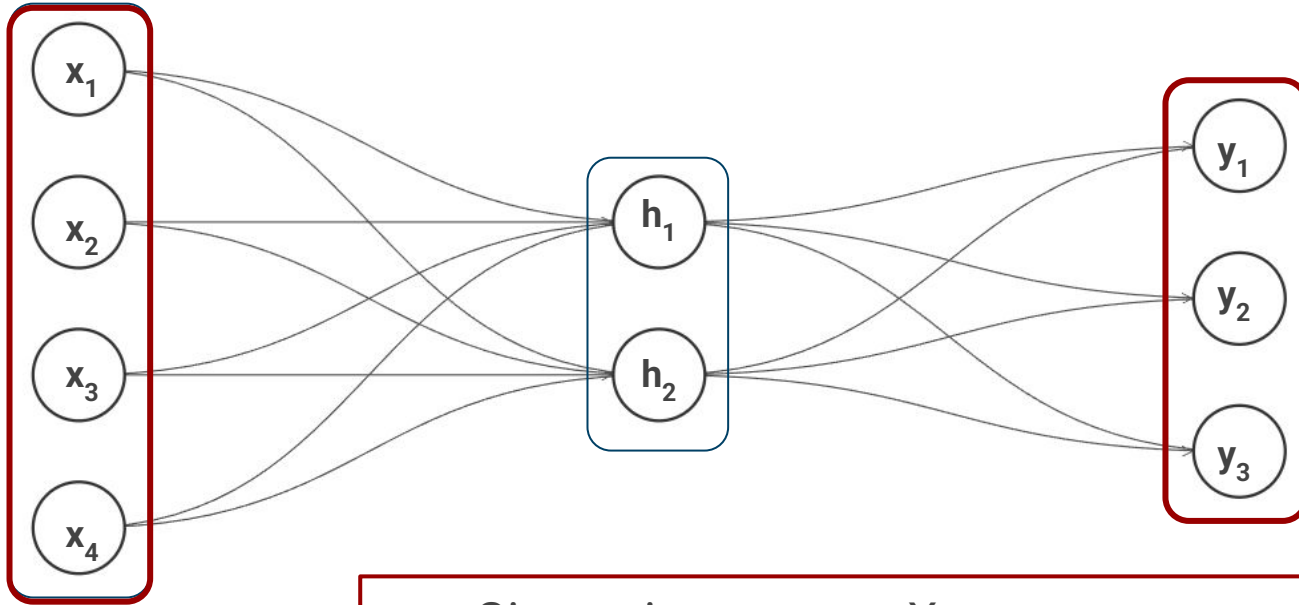
1. The function is continuous
2. The function is differentiable
3. For large positive x , the function approaches 1
4. For large negative x , the functions approaches 0
5. Summing up all softmax values is equal to 1
6. So it range between 0 and 1
7. The output can be interpreted as a probability

$$\mu(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

x	μ(x)
10	0,66394
8	0,08985
-2	0,00000
4	0,00165
-3	0,00000
9	0,24425
2	0,00022
1	0,00008
-5	0,00000
	1

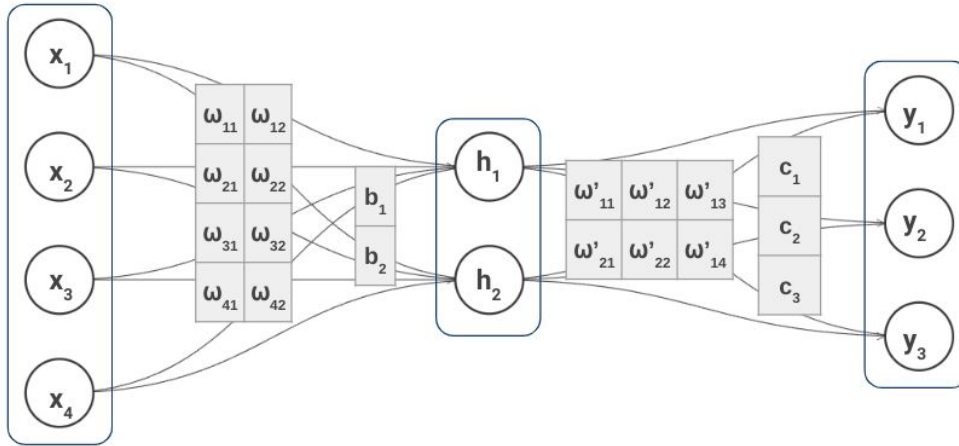
Neural Network Feed Forward

Neural Network: Feed Forwarding



Given an input vector X , we want to determine the output vector Y

Neural Network: Feed Forwarding



$$v[j] = \sum_{i=1}^C x[i]w[i, j] + b[j] \quad \forall j = 1..N$$

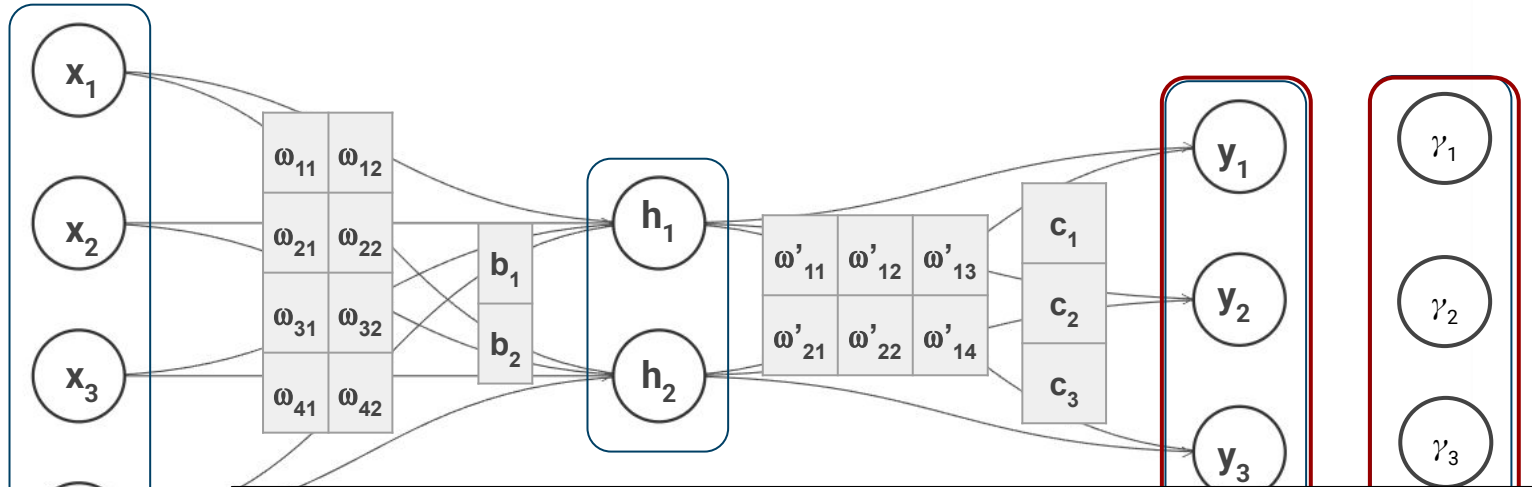
$$h[j] = \sigma(v[j]) = \frac{1}{1 + e^{-v[j]}} \quad \forall j = 1..N$$

$$u[k] = \sum_{j=1}^N h[j]w'[j, k] + c[k] \quad \forall k = 1..M$$

$$y[k] = \sigma(u[k]) = \frac{1}{1 + e^{-u[k]}} \quad \forall k = 1..M$$

Sigmoid function

Neural Network: predicted and target values



We aim to define all parameters and bias such that the **distance** between the **predicted values** and the **target value** is minimal. A **Loss Function** quantifies the error margin between a model's prediction and the actual target value

Neural Network: Loss Functions for regression

Mean Squared Error

$$L(y, \gamma) = \frac{1}{M} \sum_{k=1}^M (y[k] - \gamma[k])^2$$

This value quantifies the prediction error for a **single input**.

Neural Network: Loss Functions for classificat.

If $\mathbf{y}$ and $\boldsymbol{\gamma}$ are two **distributions**, we can use the **Cross Entropy**.

$$0 \leq \gamma[k] \leq 1 \quad \forall k = 1..M \qquad 0 \leq y[k] \leq 1 \quad \forall k = 1..M$$

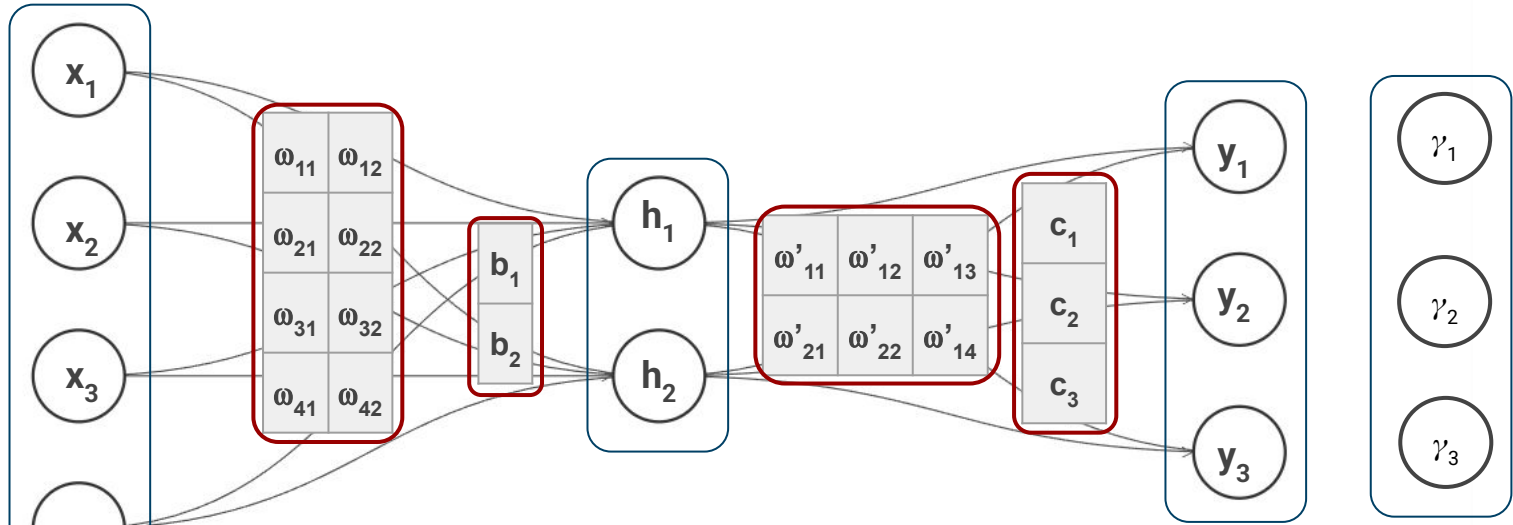
$$\sum_{k=1}^M \gamma[k] = 1$$

$$\sum_{k=1}^M y[k] = 1$$

$$L(y, \gamma) = - \sum_{k=1}^M (\gamma[k] \log_e(y[k]))$$

This is the prediction error for a **single input**

Neural Network: Stochastic Gradient Descent



Our aim is to **minimize the Error** by updating parameters. Parameters are adjusted running the **Backpropagation** phase and the **Gradient Descent**

Gradient Descent

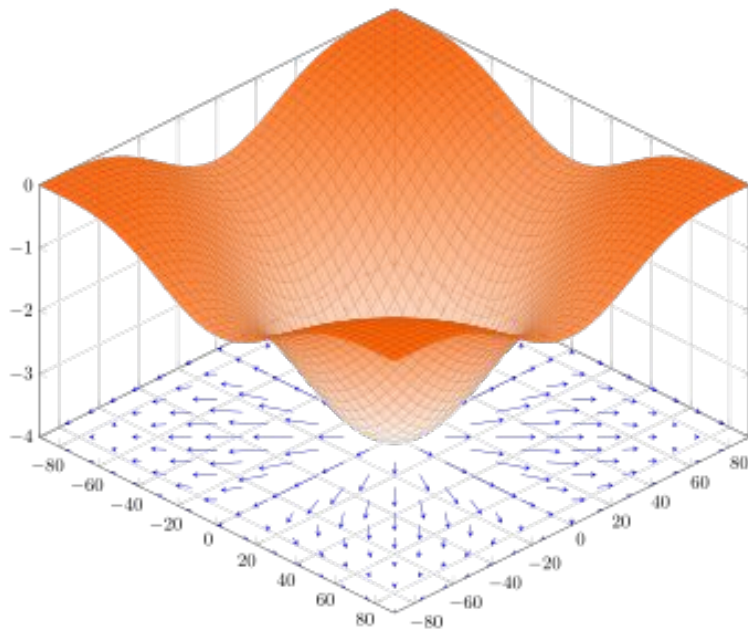
The idea behind Gradient Descent is to **compute the gradient of the loss function** with respect to the parameters of the network.

Then, we can adjust the parameters slightly **in the directions that will reduce the loss**, and **repeat the process** until we reach a selection of parameter values for which little improvement in the loss is possible.

The Gradient: from math point of view

The gradient of a function is a vector representing the direction of greatest change of the function.

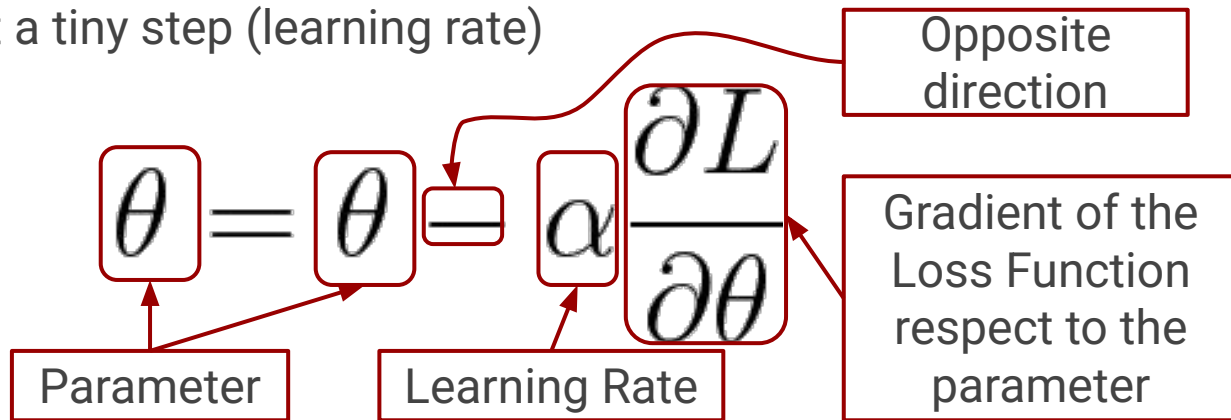
Gradients point in the direction of the steepest ascent



The Gradient: updating parameters

Once we have found the gradient of the Loss Function, we can adjust our parameters following these two rules:

- moving iteratively on the opposite direction of the gradient (toward the minimum value)
- moving for just a tiny step (learning rate)



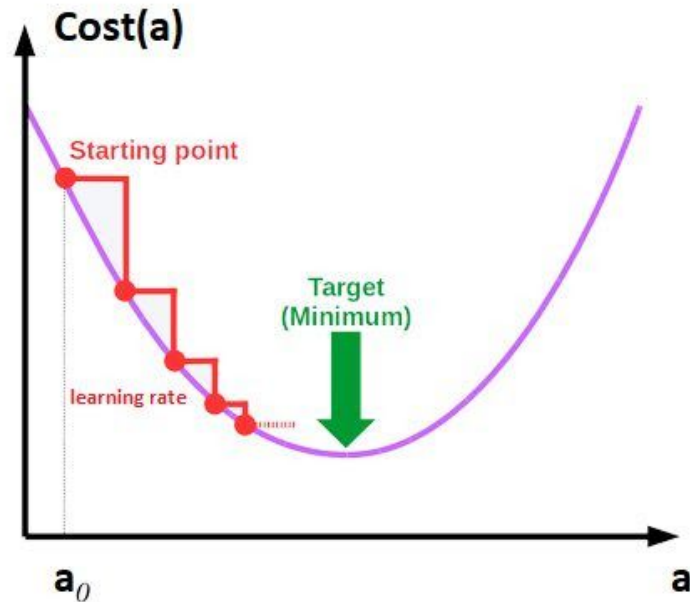
The Gradient: the learning rate

The **learning rate** is a constant that controls the step size in the parameter space. Usually is a value very little such as 0.01

By iteratively updating the model's parameters in the negative gradient direction, gradient descent gradually converges towards the optimal set of parameters that yields the lowest cost.

The Gradient: toward the minimum

After several iterations we have computed a parameter which makes our Loss Function very close to the minimum



Neural Network Back Propagation

Neural Network: BackPropagation

We need to understand how the Loss Function changes when parameters b , c , w and w' change as well.

We need to compute the following partial derivative:

$$\begin{array}{cccc}\frac{\partial L}{\partial y[k]} & \frac{\partial L}{\partial u[k]} & \frac{\partial L}{\partial c[k]} & \frac{\partial L}{\partial w'[j, k]} \\ \frac{\partial L}{\partial h[j]} & \frac{\partial L}{\partial v[j]} & \frac{\partial L}{\partial b[j]} & \frac{\partial L}{\partial w[i, j]}\end{array}$$

$$v[j] = \sum_{i=1}^C x[i]w[i, j] + b[j] \quad \forall j = 1..N$$

$$h[j] = \sigma(v[j]) = \frac{1}{1 + e^{-v[j]}} \quad \forall j = 1..N$$

$$u[k] = \sum_{j=1}^N h[j]w'[j, k] + c[k] \quad \forall k = 1..M$$

$$y[k] = \sigma(u[k]) = \frac{1}{1 + e^{-u[k]}} \quad \forall k = 1..M$$

$$L(y, \gamma) = \frac{1}{M} \sum_{k=1}^M (y[k] - \gamma[k])^2$$

Neural Network: BackPropagation

$$\frac{\partial L}{\partial y[k]} = \frac{2}{M} (y[k] - \gamma[k])$$

$$v[j] = \sum_{i=1}^C x[i]w[i, j] + b[j] \quad \forall j = 1..N$$

$$h[j] = \sigma(v[j]) = \frac{1}{1 + e^{-v[j]}} \quad \forall j = 1..N$$

$$u[k] = \sum_{j=1}^N h[j]w'[j, k] + c[k] \quad \forall k = 1..M$$

$$y[k] = \sigma(u[k]) = \frac{1}{1 + e^{-u[k]}} \quad \forall k = 1..M$$

$$L(y, \gamma) = \frac{1}{M} \sum_{k=1}^M (y[k] - \gamma[k])^2$$

Neural Network: BackPropagation

$$\frac{\partial L}{\partial u[k]} = \frac{\partial L}{\partial y[k]} \frac{\partial y[k]}{\partial u[k]}$$

Chain Rule

$$= \frac{2}{M} (y[k] - \gamma[k]) y[k] (1 - y[k])$$

Remind that: $\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$

and

$$\mathbf{y}[\mathbf{k}] = \sigma(\mathbf{u}[\mathbf{k}])$$

$$v[j] = \sum_{i=1}^C x[i] w[i, j] + b[j] \quad \forall j = 1..N$$

$$h[j] = \sigma(v[j]) = \frac{1}{1 + e^{-v[j]}} \quad \forall j = 1..N$$

$$u[k] = \sum_{j=1}^N h[j] w'[j, k] + c[k] \quad \forall k = 1..M$$

$$y[k] = \sigma(u[k]) = \frac{1}{1 + e^{-u[k]}} \quad \forall k = 1..M$$

$$\frac{\partial L}{\partial y[k]} = \frac{2}{M} (y[k] - \gamma[k])$$

Neural Network: BackPropagation

When should the BackPropagation stop?

1. Reaching a maximum number of iterations
2. After a predetermined number of training epochs (an epoch is a complete pass through the training dataset)
3. Convergence of the Loss: when the change in the loss between consecutive epochs becomes smaller than a predefined threshold: $|L^{(t)} - L^{(t-1)}| < \epsilon$

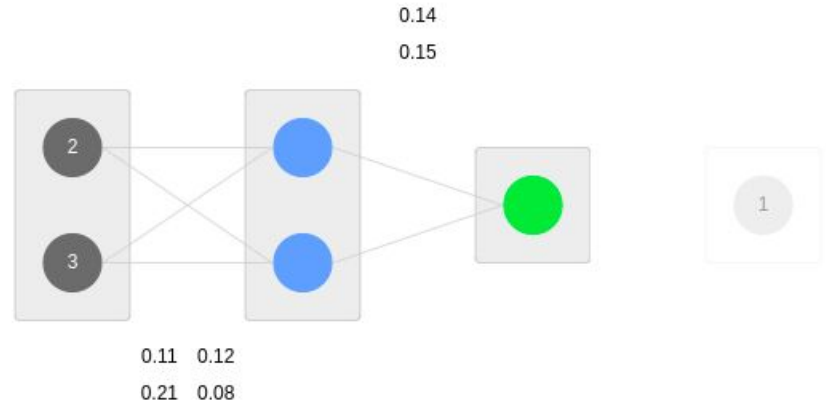
Neural Network Playground

Neural Network Online Simulator

NetFlowJS:

<https://hmkcode.com/netflow/>

You can see visualization of the forward pass and backpropagation



Word Embedding and Word2Vec

Word Embedding

If I say “Ruler” or “Worker”, which one of these words is more similar to “King”?

Perhaps many people say “Ruler”, because it makes more sense.

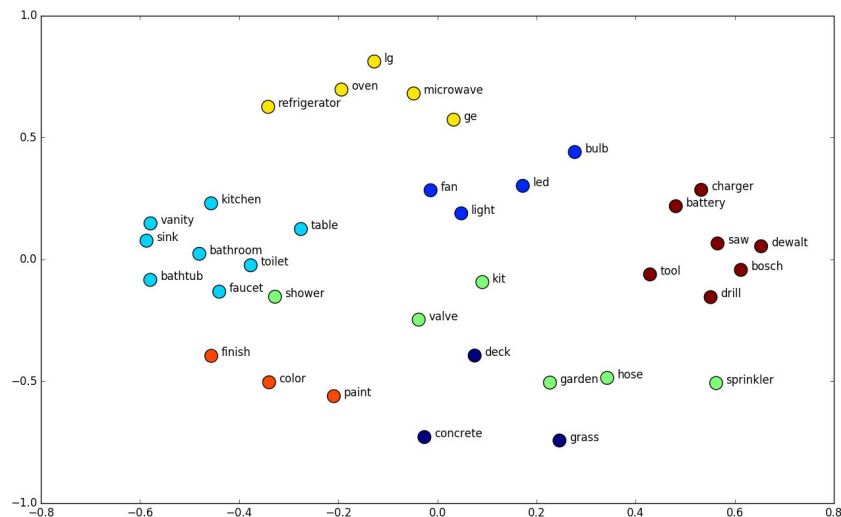
But how can we teach this intuition to a computer?

That’s where word embeddings come in handy

Word Embedding

A word embedding is a **representation of a word**

A word is represented by **real-valued vector**, which encodes the meaning of the word in such a way that the **words that are closer in the vector space are expected to be similar in meaning**.



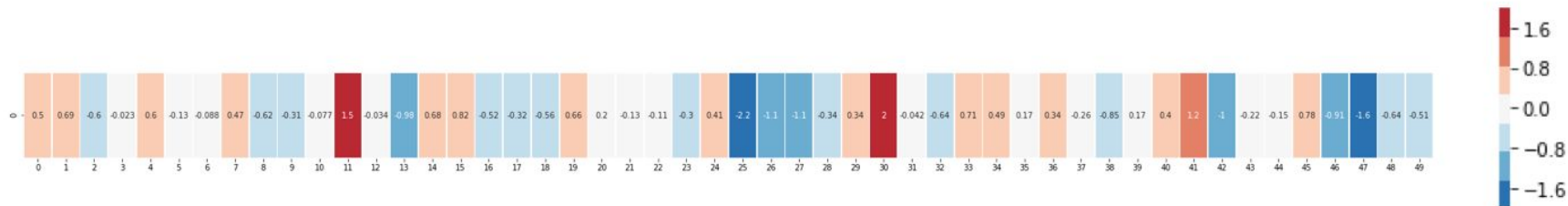
Word Embedding

Word Embedding is used very often:

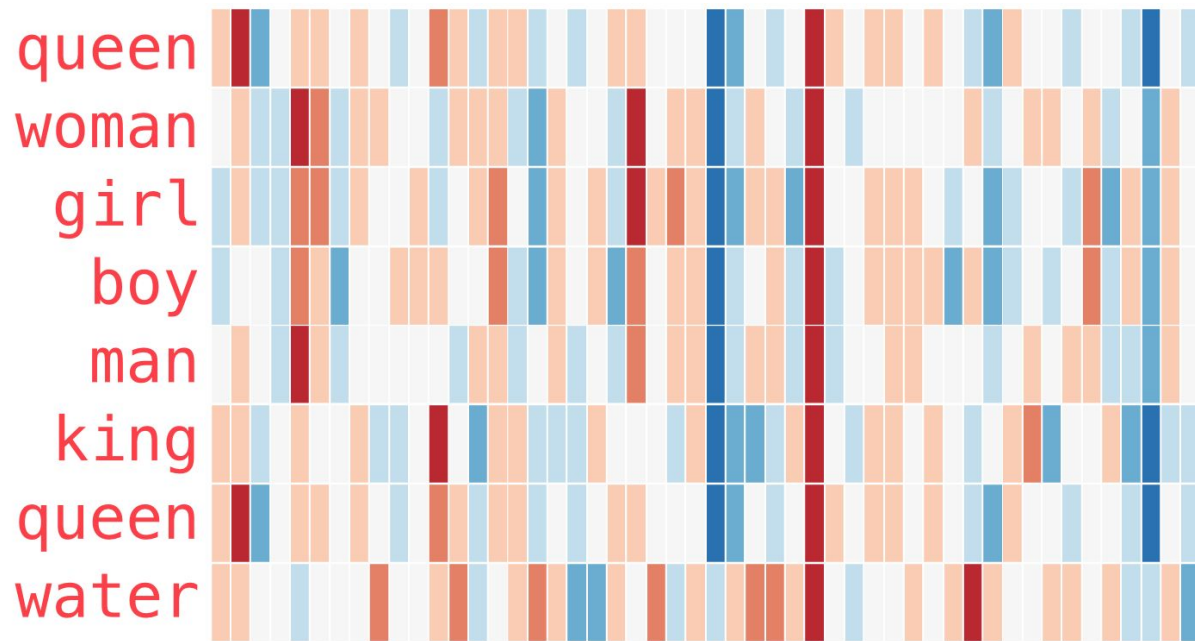
- Syntactic Parsing
- Sentiment Analysis
- Document Clustering
- Text Classification
- Next Word Prediction

The vector for the word “King”

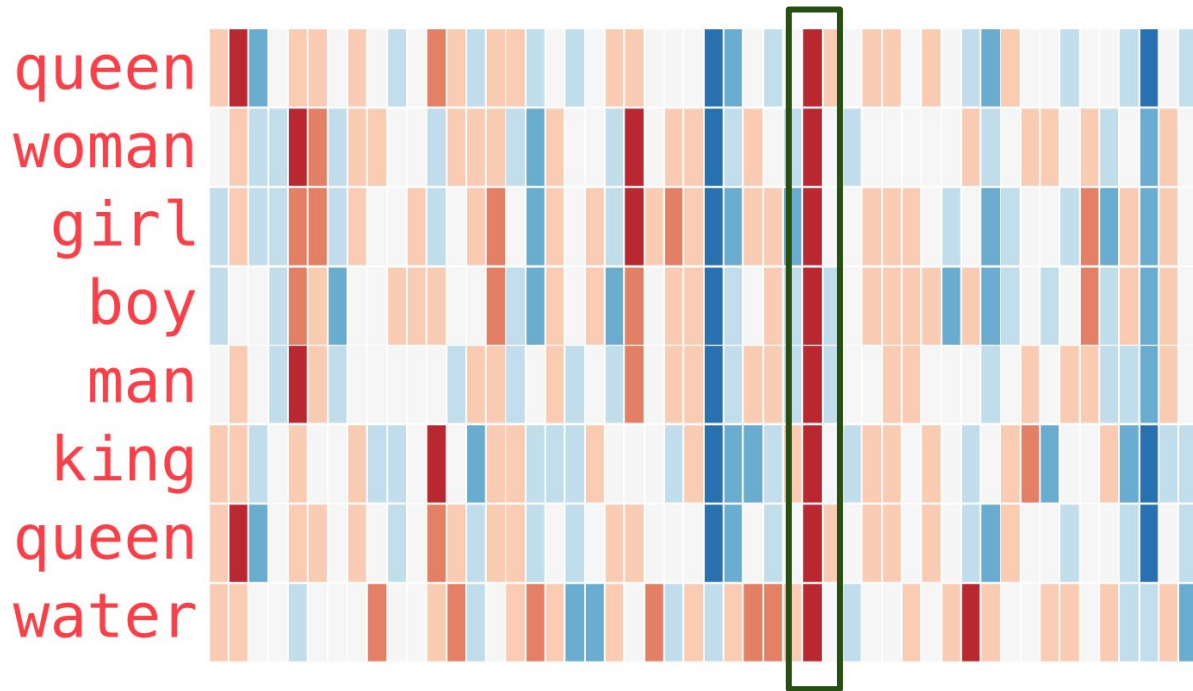
```
[ 0.50451 , 0.68607 , -0.59517 , -0.022801, 0.60046 , -0.13498 , -0.08813 , 0.47377 , -0.61798 , -0.31012 ,  
-0.076666, 1.493 , -0.034189, -0.98173 , 0.68229 , 0.81722 , -0.51874 , -0.31503 , -0.55809 , 0.66421 , 0.1961  
, -0.13495 , -0.11476 , -0.30344 , 0.41177 , -2.223 , -1.0756 , -1.0783 , -0.34354 , 0.33505 , 1.9927 ,  
-0.04234 , -0.64319 , 0.71125 , 0.49159 , 0.16754 , 0.34344 , -0.25663 , -0.8523 , 0.1661 , 0.40102 , 1.1685 ,  
-1.0137 , -0.21585 , -0.15155 , 0.78321 , -0.91241 , -1.6106 , -0.64426 , -0.51042 ]
```



Vectors for eight words

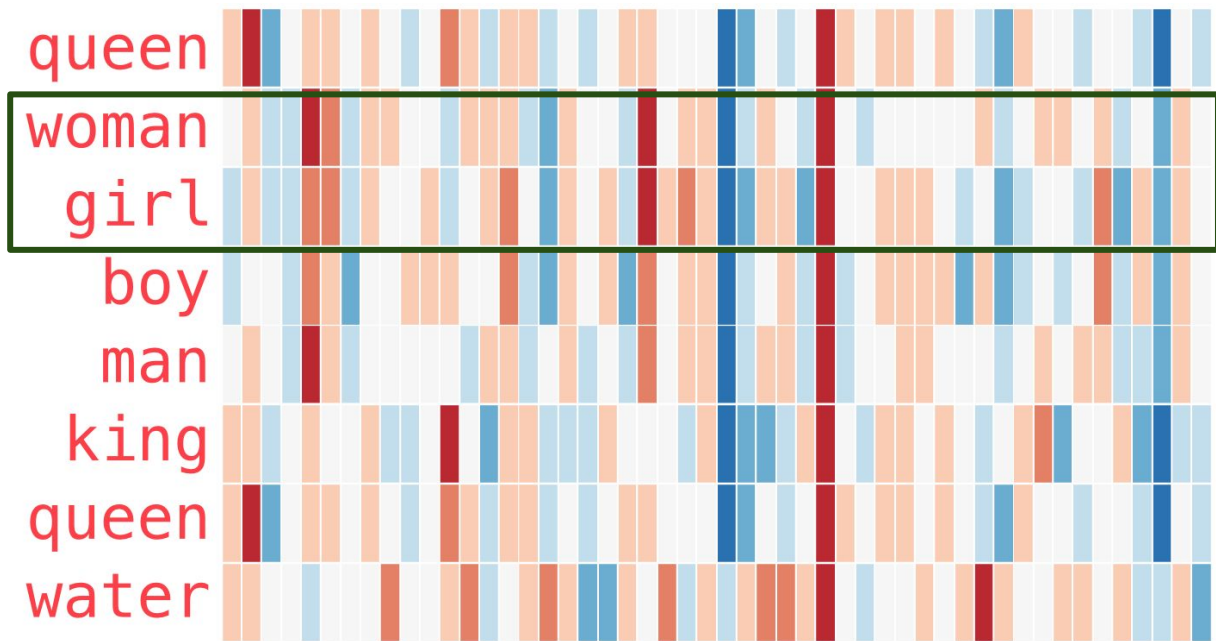


Vectors for eight words



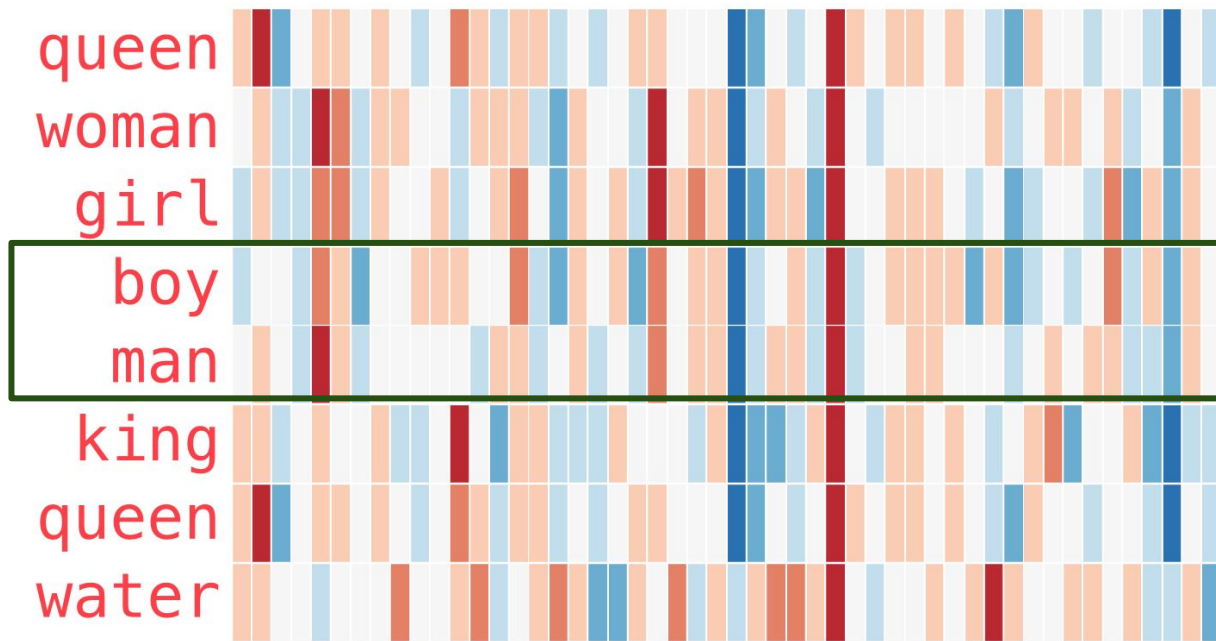
All words are similar along this dimension (we don't know what each dimension codes for)

Vectors for eight words



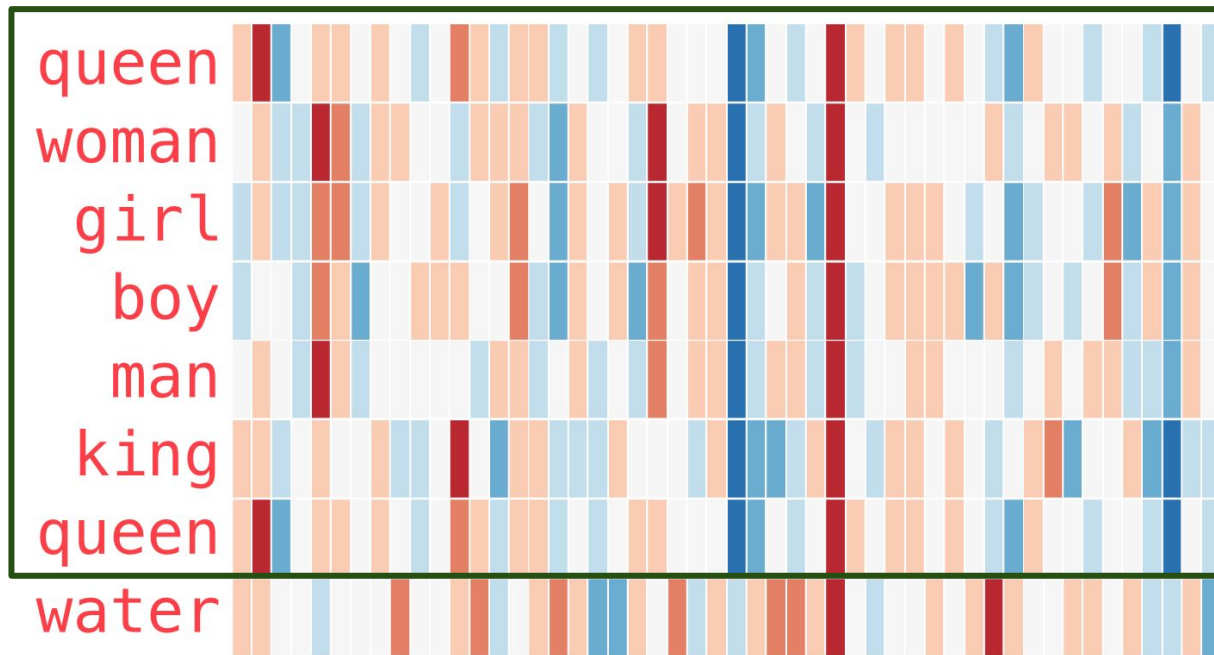
Woman and Girl are very similar to each other in a lot of dimensions

Vectors for eight words



Even Boy and Man are very similar to each other in a lot of dimensions

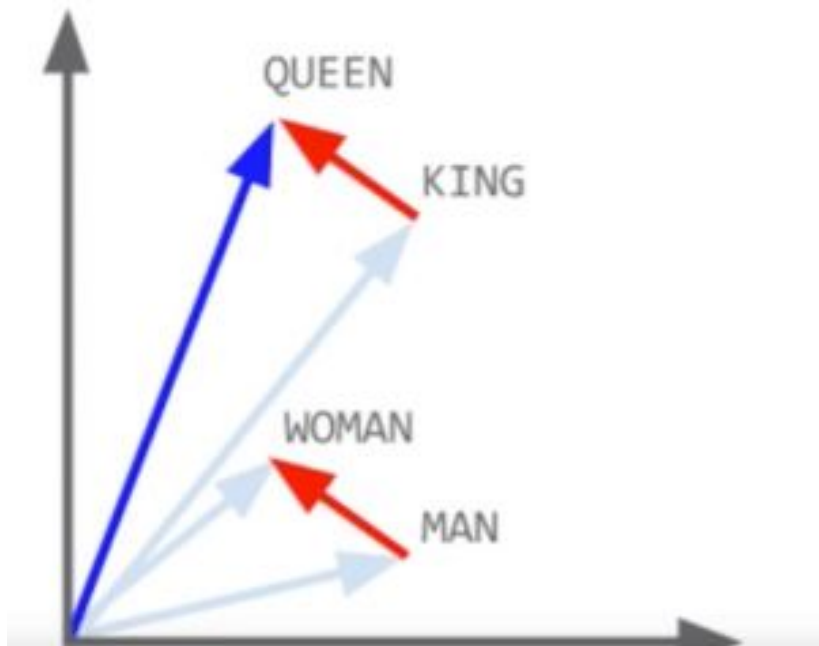
Vectors for eight words



All vectors representing people share many dimensions. They seem to be belonging to the same category.

Water instead seems to be very different to the others

What can we use vectors for?



Using vectors we can use classical vector operation such as sum and difference.

For example we can see that the difference between the vectors representing KING and QUEEN is very similar to the difference between MAN and WOMAN

king - man + woman $\approx$ queen



Adding and Subtracting Vectors

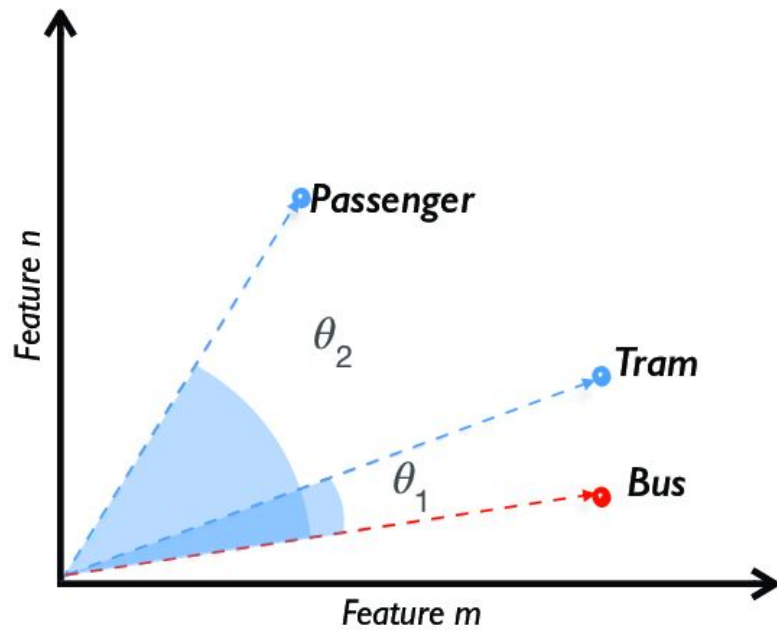
We can also use arithmetic on these embeddings to **derive meaning**.

For instance: “Berlin-Germany+France=Paris”

Similarity between words

Representing words by vectors, we can also compute similarity between words.

Usually **Cosine Similarity** is used



Two different paradigms for embeddings

1. Count-based paradigm

- a. Up until 2013, the traditional models for NLP tasks were count-based models. They mainly involve computing a **co-occurrence matrix** to capture meaningful relationships among words.

2. Prediction-based paradigm

- a. this model uses **neural networks** to learn representation of words in a vector space

Count-Based Paradigm

Doc 1: *all that glitters is not gold*

Doc 2: *all is well that ends well*

*	START	all	that	glitters	is	not	gold	well	ends	END
START	0	2	0	0	0	0	0	0	0	0
all	2	0	1	0	1	0	0	0	0	0
that	0	1	0	1	0	0	0	1	1	0
glitters	0	0	1	0	1	0	0	0	0	0
is	0	1	0	1	0	1	0	1	0	0
not	0	0	0	0	1	0	1	0	0	0
gold	0	0	0	0	0	1	0	0	0	1
well	0	0	1	0	1	0	0	0	1	1
ends	0	0	1	0	0	0	0	1	0	0
END	0	0	0	0	0	0	1	1	0	0

Count-Based Paradigm

- Square Matrix $V \times V$
- V is the size of the vocabulary
- 2 special tokens are added (START and END) to represent the beginning and end of sentences, paragraphs or documents
- Symmetric
- Sparse (too many zeros)

*	START	all	that	glitters	is	not	gold	well	ends	END
START	0	2	0	0	0	0	0	0	0	0
all	2	0	1	0	1	0	0	0	0	0
that	0	1	0	1	0	0	0	1	1	0
glitters	0	0	1	0	1	0	0	0	0	0
is	0	1	0	1	0	1	0	1	0	0
not	0	0	0	0	1	0	1	0	0	0
gold	0	0	0	0	0	1	0	0	0	1
well	0	0	1	0	1	0	0	0	1	1
ends	0	0	1	0	0	0	0	1	0	0
END	0	0	0	0	0	0	1	1	0	0

Count-Based Paradigm

- The word *is* appears one time next to the word *all*, *glitters*, *not*, *well*
- The word *gold* never appears neither next to the word *glitters* nor to the word *ends*
- Related words are observed (counted) together more often than unrelated words

*	START	all	that	glitters	is	not	gold	well	ends	END
START	0	2	0	0	0	0	0	0	0	0
all	2	0	1	0	1	0	0	0	0	0
that	0	1	0	1	0	0	0	1	1	0
glitters	0	0	1	0	1	0	0	0	0	0
is	0	1	0	1	0	1	0	1	0	0
not	0	0	0	0	1	0	1	0	0	0
gold	0	0	0	0	0	1	0	0	0	1
well	0	0	1	0	1	0	0	0	1	1
ends	0	0	1	0	0	0	0	1	0	0
END	0	0	0	0	0	0	1	1	0	0

Count-Based Paradigm

- This is a 10D vector representing the word *glitters*
- The vector represents the position of the word in ten-dimensional space

*	START	all	that	glitters	is	not	gold	well	ends	END
START	0	2	0	0	0	0	0	0	0	0
all	2	0	1	0	1	0	0	0	0	0
that	0	1	0	1	0	0	0	1	1	0
glitters	0	0	1	0	1	0	0	0	0	0
is	0	1	0	1	0	1	0	1	0	0
not	0	0	0	0	1	0	1	0	0	0
gold	0	0	0	0	0	1	0	0	0	1
well	0	0	1	0	1	0	0	0	1	1
ends	0	0	1	0	0	0	0	1	0	0
END	0	0	0	0	0	0	1	1	0	0

Count-Based Paradigm

When V is too large, the matrix become unmanageable.

In most NLP tasks, the co-occurrence matrix goes through PCA or SVD for **dimensionality reduction** and decomposed into a new k -dimensional matrix

*	k_1	k_2
START	0.705	0.484
all	0.705	-0.484
that	0.654	-0.783
glitters	0.52	0
is	1.027	0
not	0.654	0.783
gold	0.382	0.656
well	0.382	-0.656
ends	1.394	-1.061
END	1.394	1.061

Toward Prediction-based Paradigm

Many attempts were made to improve the performance of this model, but the model did not do well in capturing complex relationships among words.

In 2013, Thomas Mikolov proposed the prediction-based modeling technique, called Word2Vec.

The model uses **neural networks** to learn intelligent representations of words in a vector space.

Word2Vec

Word2Vec is a method to efficiently create word embeddings and has been founded by Google in 2013.

Word2Vec is a technique used to convert words into **multidimensional vectors**

Word2Vec takes a large **corpus of text as input** and generates a vector space with **hundreds of dimensions**. Each unique word in the corpus is assigned a vector in this space.

What is behind Word2Vec?

Word2Vec is trained using a neural network.

How is Word2Vec trained?

The tool defines two training models:

1. Continuous Bag Of Words (CBow)
2. Skip-Gram

One-hot encoding vector

	1	2	3	4	5	6	7	8
I	1	0	0	0	0	0	0	0
ate	0	1	0	0	0	0	0	0
an	0	0	1	0	0	0	0	0
apple	0	0	0	1	0	0	0	0
and	0	0	0	0	1	0	0	0
played	0	0	0	0	0	1	0	0
the	0	0	0	0	0	0	1	0
piano	0	0	0	0	0	0	0	1

Both CBoW and Skip Gram takes into account words represented by **one-hot encoding vectors**.

A one-hot encoding vector is a vector having size V (the number of words in the vocabulary). There are just a single 1 and $V-1$ zeros.

In the example, $[0, 0, 0, 1, 0, 0, 0, 0]$ is the word **apple**

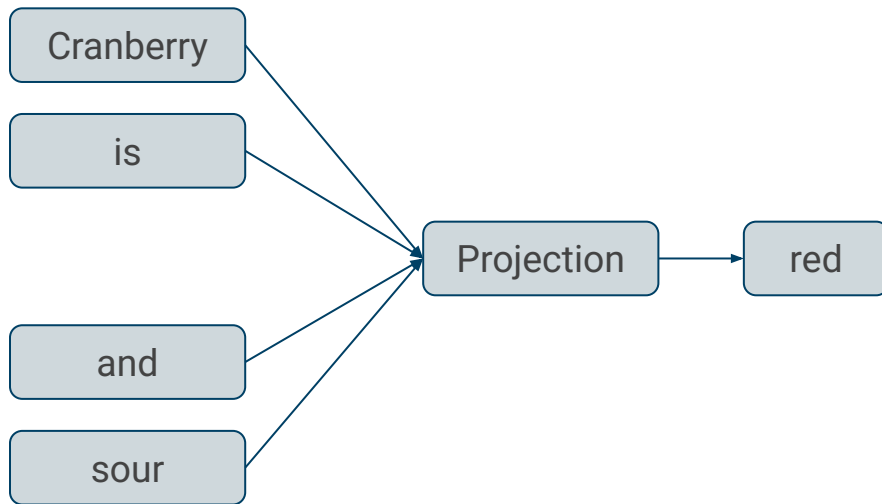
Continuous Bag Of Words

Continuous Bag Of Words

This model predicts the **center word** using the surrounding **context words**, which comprises a few words before and after the current word

You can think of CBOW like the “fill the gap” game.

Cranberry is ____ and sour



Continuous Bag Of Words: Sliding Window



Text is processed using a **Sliding Window**.

In this example, the **window size** is 2 (two words before the center word and two after)

Red word is the **center word**, whilst blue words are the **context words**.

Continuous Bag Of Words: Sliding Window



CBOW predicts the red word given the context (blue words)

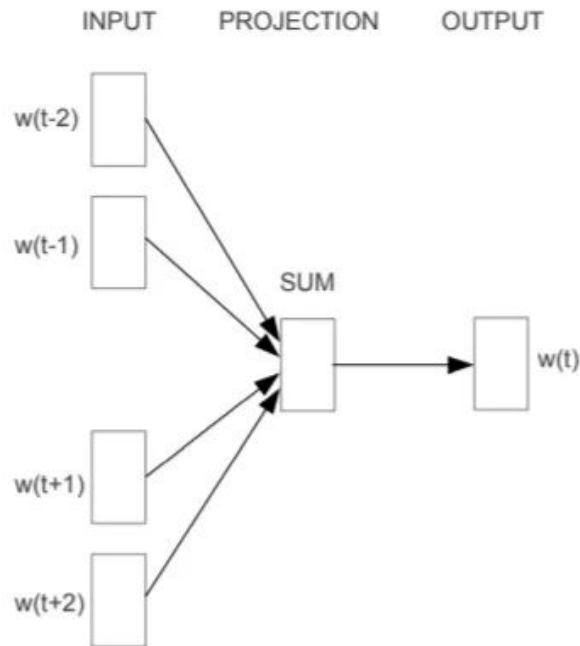
The size of the sliding window is a **hyper-parameter** and usually ranges between 1 and 10

Continuous Bag Of Words

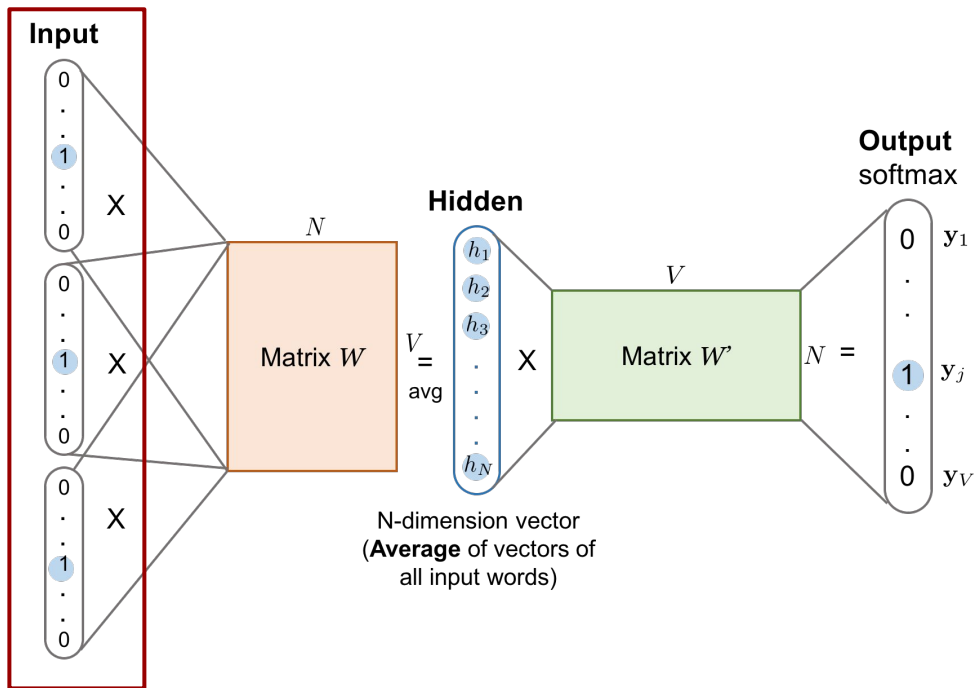
The CBOW model is based on a **Neural Network** consisting of an input layer, a hidden layer, and an output layer.

The input layer is used to represent the **context words**.

The output layer is used to predict the **center word**



Continuous Bag Of Words



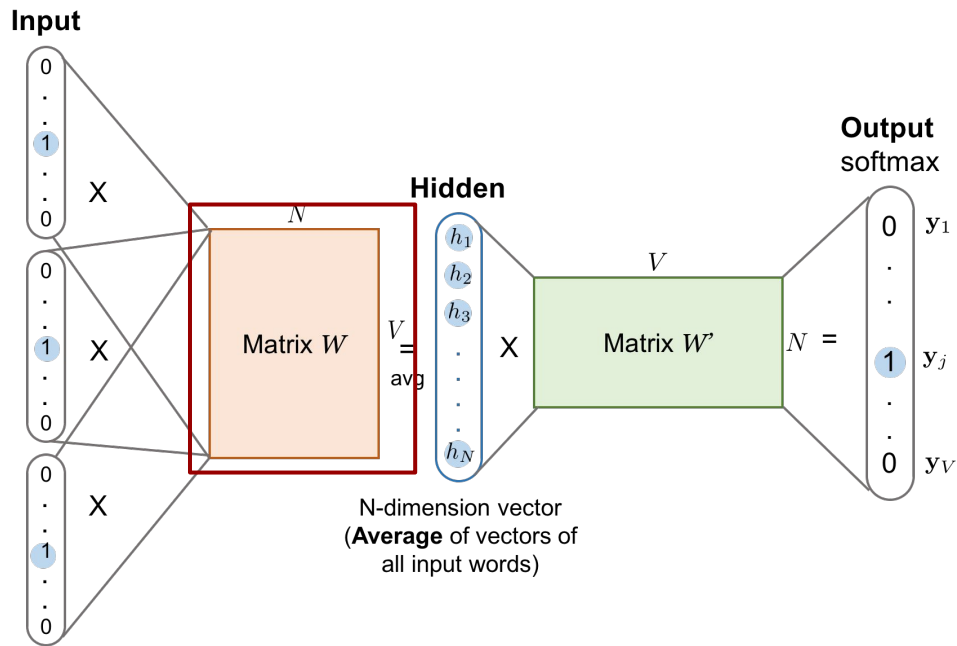
All the m surrounding context words are represented by V -dimension vectors.

V is the size of the vocabulary

M is the number of words in the context

Input vectors are **one-hot encoded vectors** ($V-1$ zeros, only one 1)

Continuous Bag Of Words

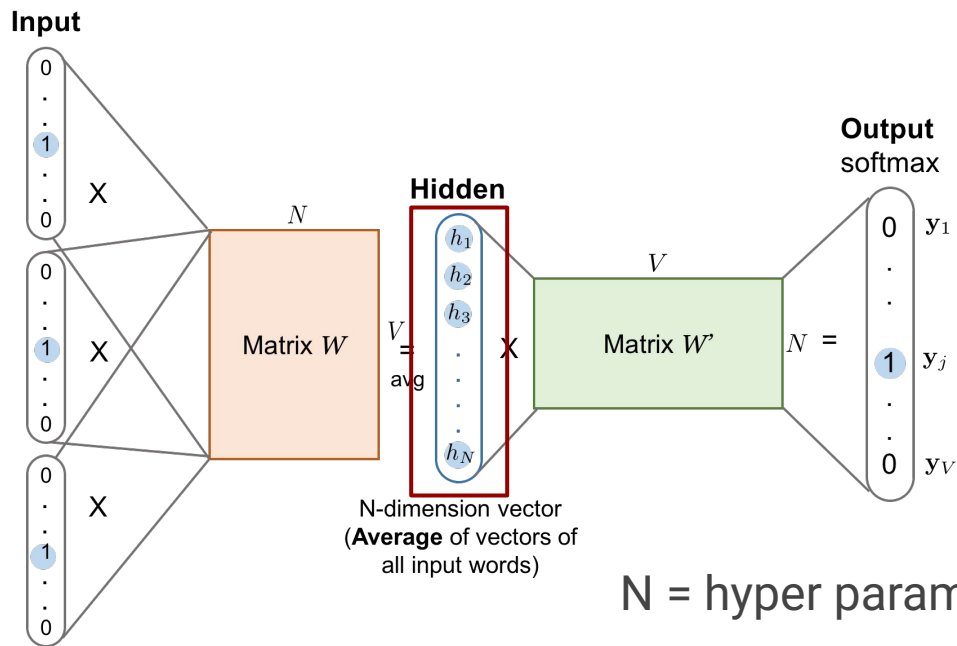


Matrix W is a $V \times N$ matrix representing the **context embedding matrix**

All the elements in W matrix are the first set of parameters that our Neural Network must learn, Usually it is initialized **randomly**

N is an hyper-parameter

Continuous Bag Of Words



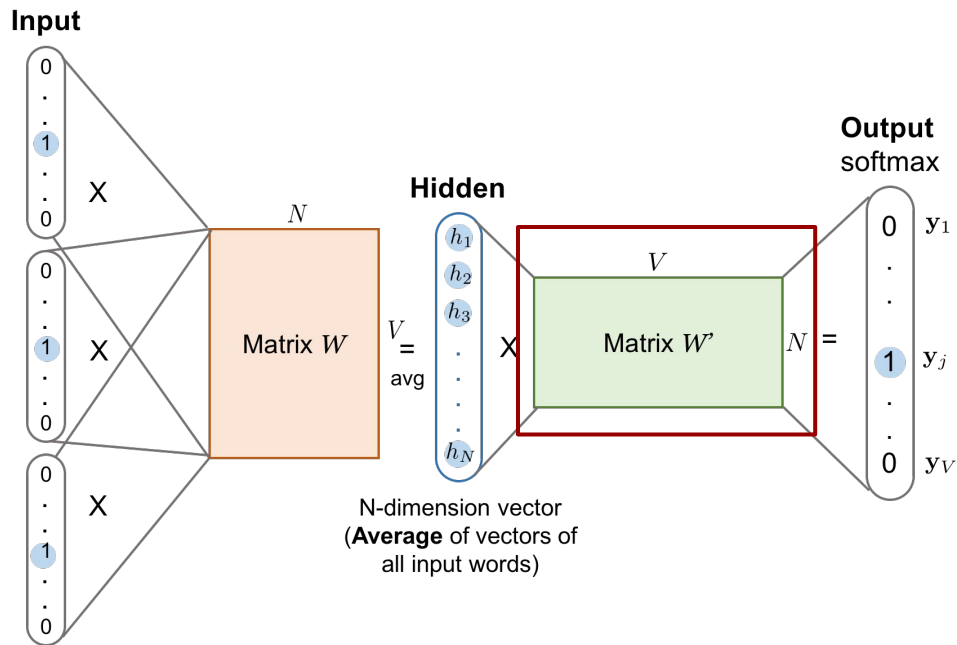
The hidden layer is a N -dimension vector which is computed by multiplying all input vectors by the matrix W , then computing the average.

$$h[j] = \frac{1}{M} \sum_{m=1}^M \sum_{i=1}^V x_m[i] W[i, j]$$

N = hyper parameter; V = Vocabulary size;

M = number of word i in the context

Continuous Bag Of Words

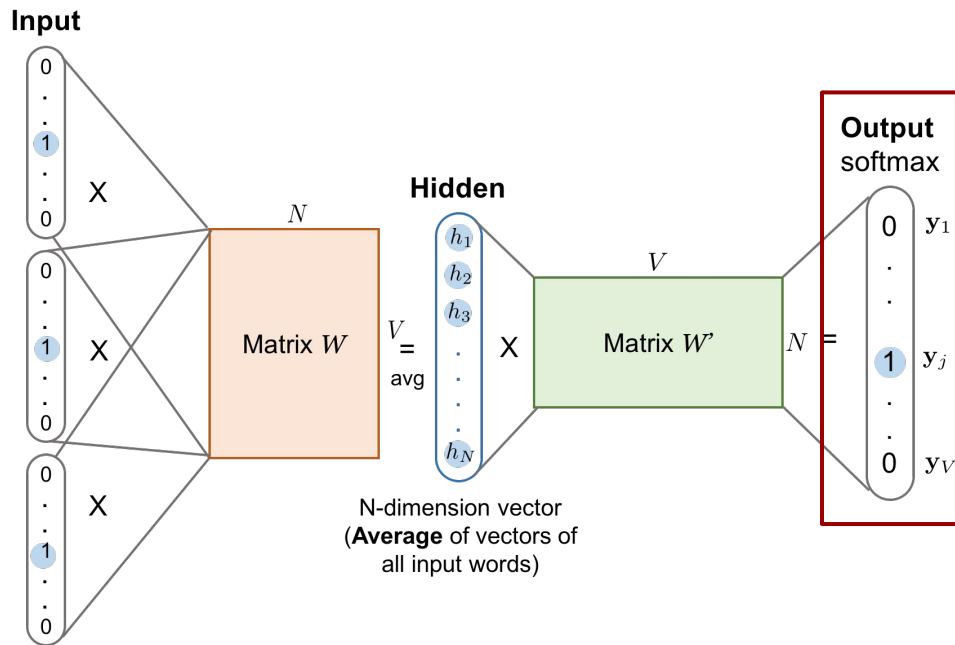


Matrix W' is a $N \times V$ matrix representing the **word embedding matrix**.

All the elements in W' matrix are the second set of parameters that our Neural Network must learn. Usually it is initialized **randomly**

N is an hyper-parameter

Continuous Bag Of Words

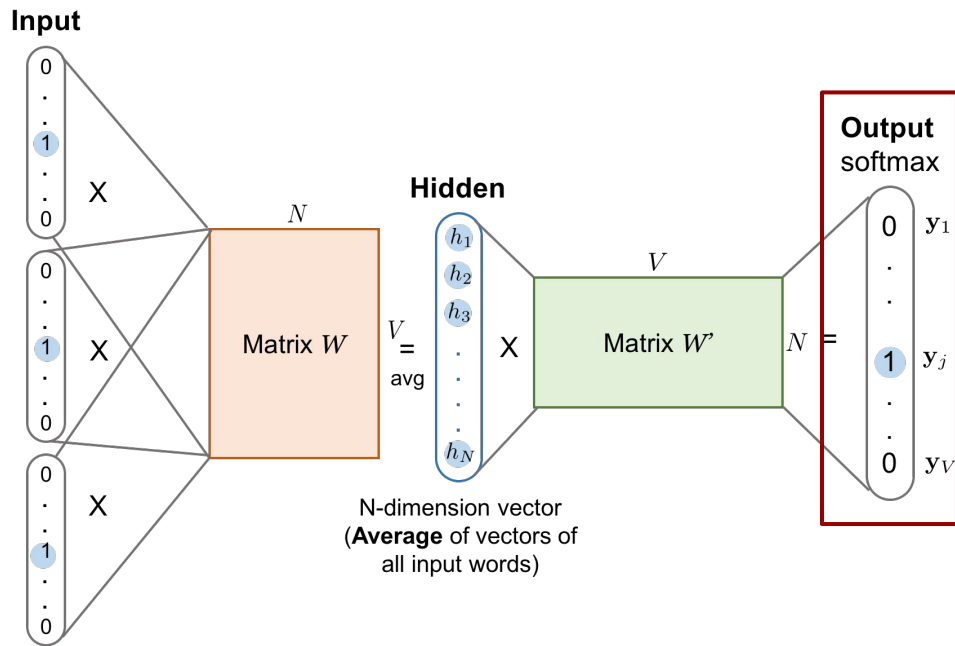


Predicted center word is represented by V -dimension vectors.

V is the size of the vocabulary

This vector is computed by multiplying the hidden vector by the matrix W' and applying the **Softmax Activation Function**

Continuous Bag Of Words

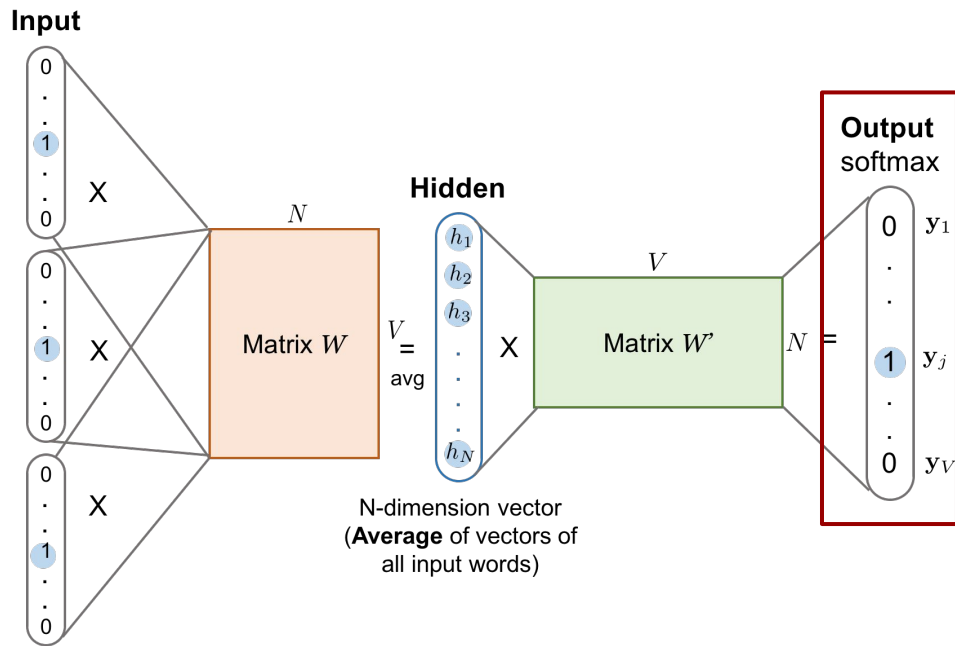


$$u[k] = \sum_{j=1}^N h[j]W'[j, k]$$

Softmax Activation Function

$$y[k] = \mu(u[k]) = \frac{e^{u[k]}}{\sum_{l=1}^V e^{u[l]}}$$

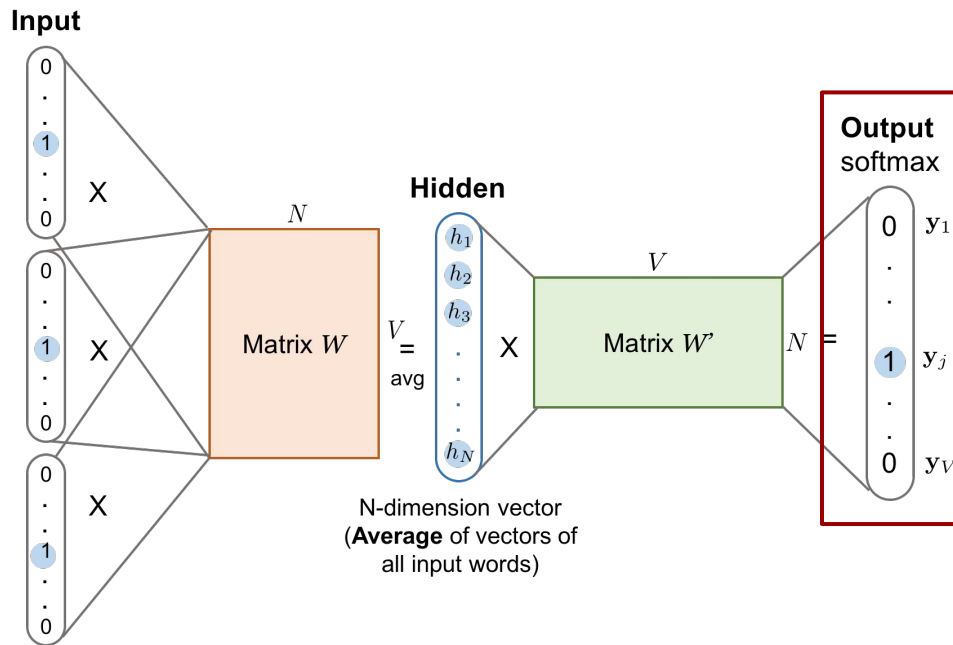
Continuous Bag Of Words



Ideally, we would like to have an output vectors like **one-hot encoded vector** ($V-1$ zeros, only one 1)

In reality, each element is a real value belonging to $[0, 1]$ representing the **conditional probability** for each word of being the center word.

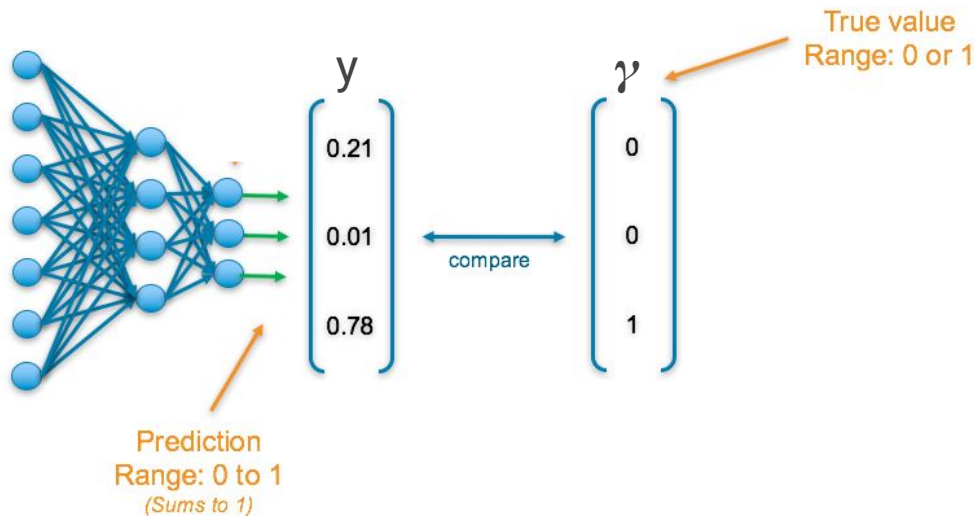
Continuous Bag Of Words



The sum of all elements belonging to the predicted vector must be **equal to 1**

The target word is selected as the word with the **highest probability** in y .

Continuous Bag Of Words: Error

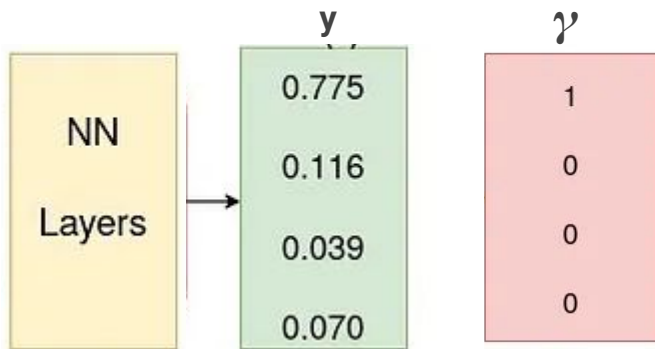


The prediction is not always the same as the the actual value.

The distance between the predicted value and the actual value is the **error**.

We would like to have a minimum error. So we need to define a **Loss Function**

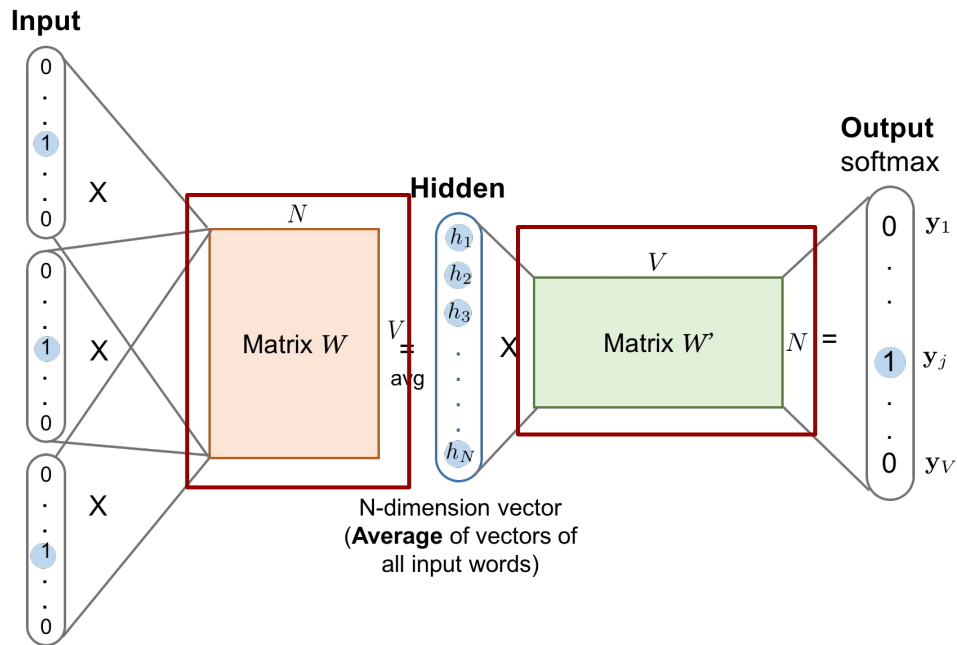
Continuous Bag Of Words: Cross Entropy



Both y and γ can be considered as **distributions**, so we can use the **Cross Entropy**.

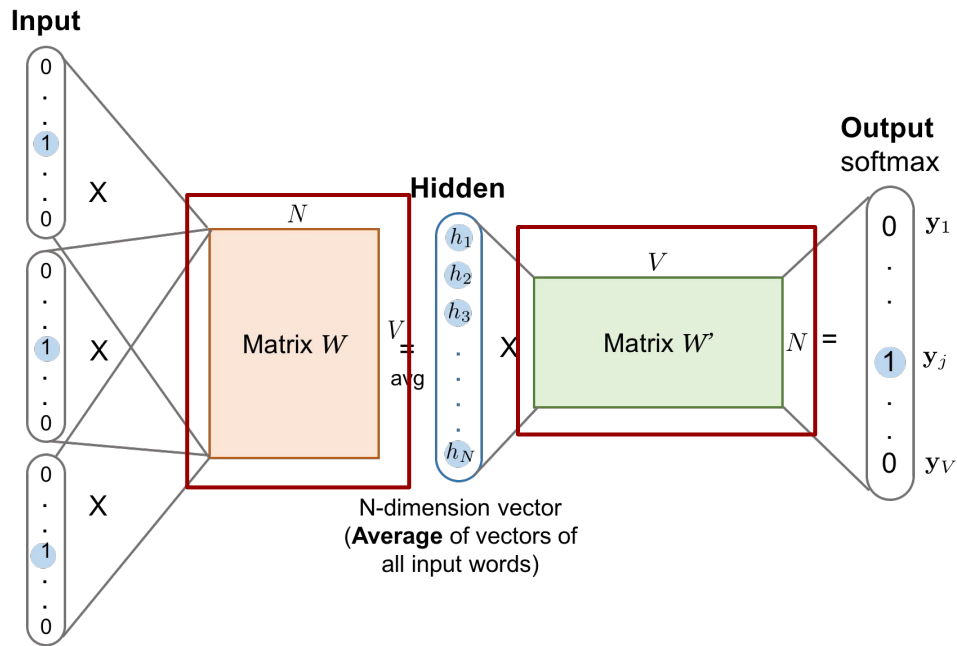
$$L(y, \gamma) = - \sum_{k=1}^V \gamma[k] \log_e(y[k])$$

Continuous Bag Of Words



To **reduce the error** between the predicted center word and the real center word, Matrices W and W' are optimized through **backward** propagations

Continuous Bag Of Words



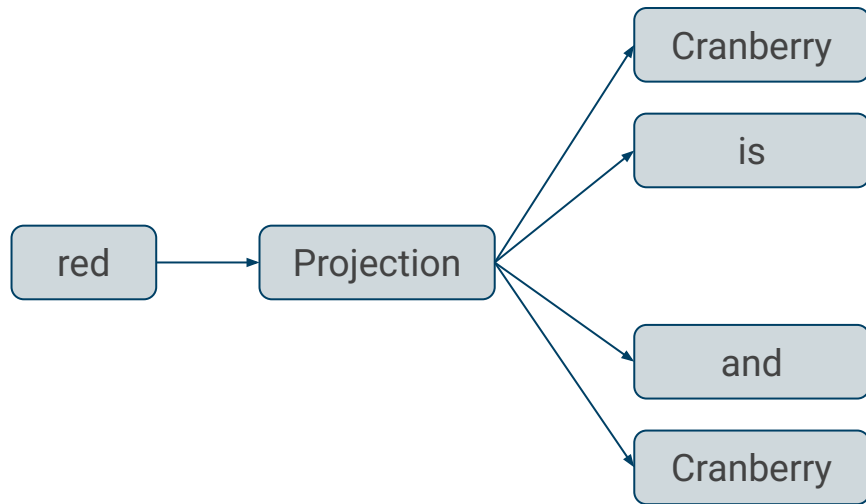
Parameters are adjusted running the Backpropagation phase and the **Stochastic Gradient Descent**

Skip-Gram

Skip-Gram

This model is the opposite of CBOW.

It aims to predict the surrounding **context words** given the **center word**.



Skip-Gram: Sliding Window



Even in Skip-Gram, text is processed using a **Sliding Window**.

In this example, the window size is 2 (two words before the center word and two after)

Red word is the **center word**, whilst blue words are the **context words**.

Skip-Gram: Sliding Window



Skip-Gram predicts the blue words given the center word (red word)

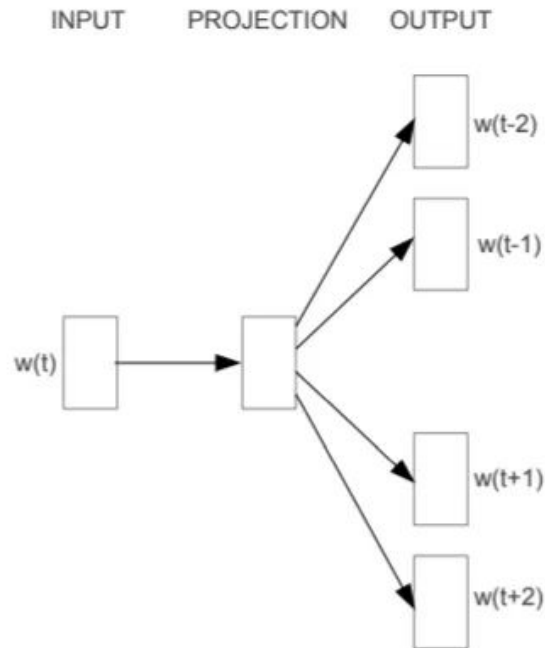
The size of the sliding window is a hyper-parameter and usually ranges between 1 and 10

Skip-Gram

Even the Skip-Gram model is based on a **Neural Network** consisting of an input layer, a hidden layer, and an output layer.

The input layer is used to represent the **center word**.

The output layer is used to predict the **surrounding context words**

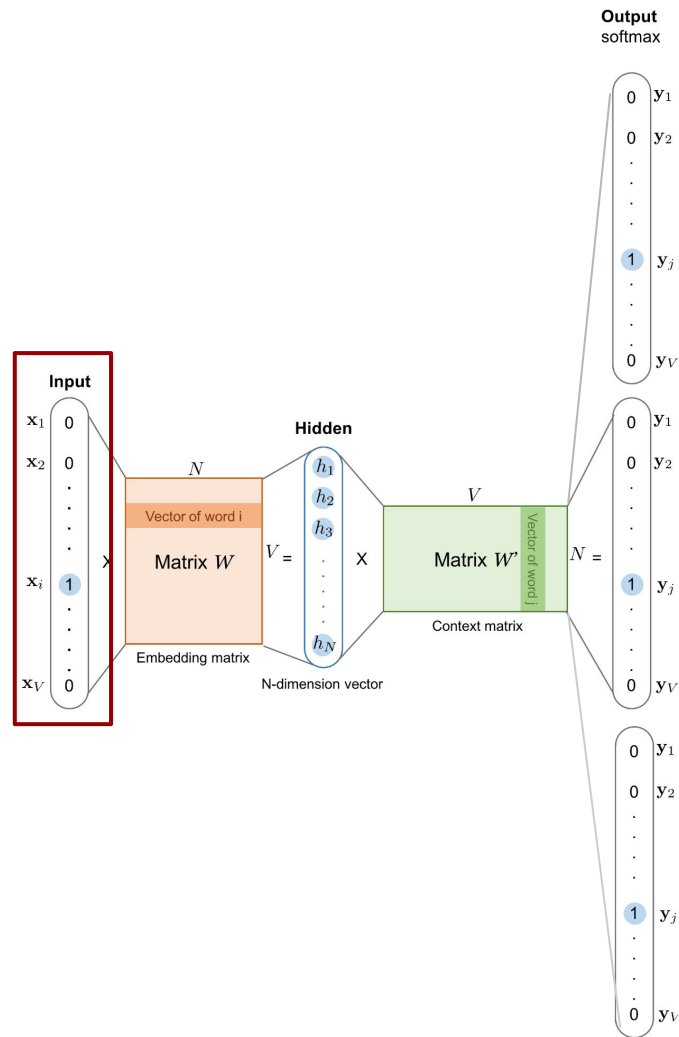


Skip-Gram

Opposite of CBOW:

One **one-hot encoded** V -dimension vector for the center word ($V-1$ zeros, only one 1)

V is the size of the vocabulary

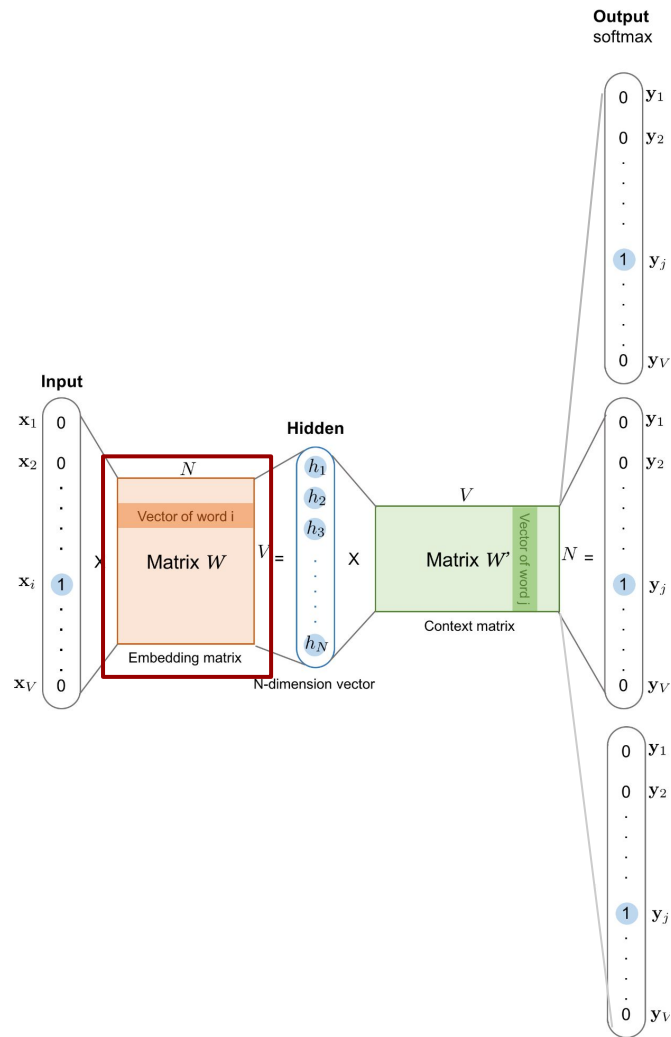


Skip-Gram

Matrix W is a $V \times N$ matrix representing the **word embedding matrix**

All the elements in W matrix are the first set of parameters that our Neural Network must learn. Usually it is initialized **randomly**

N is an hyper-parameter.



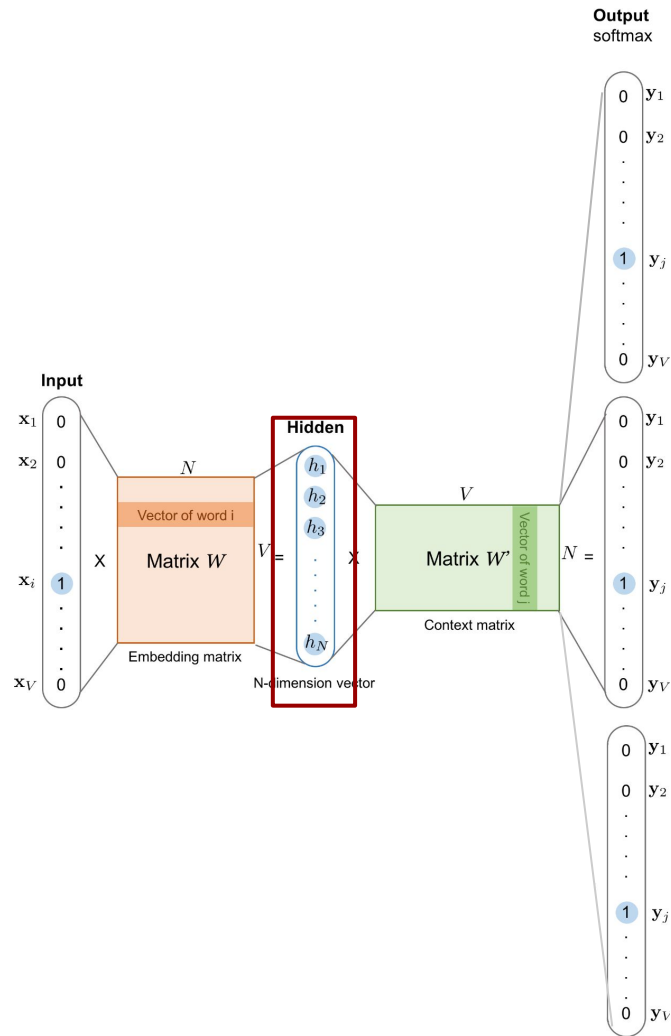
Skip-Gram

The hidden layer is a N-dimension vector which is computed by multiplying the input vector by the matrix W.

$$h[j] = \sum_{i=1}^V x[i]W[i, j]$$

V = Vocabulary Size

N = Hyper Parameter

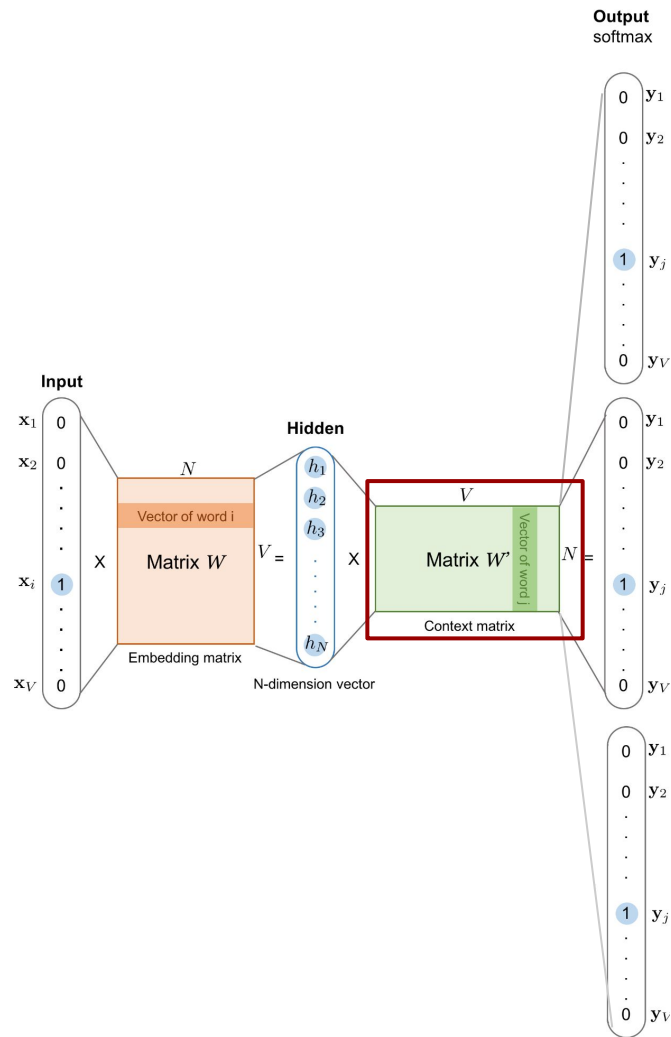


Skip-Gram

Matrix W' is a $N \times V$ matrix representing the **Context Matrix**.

All the elements in W' matrix are the second set of parameters that our Neural Network must learn. Usually it is initialized **randomly**

N is an hyper-parameter

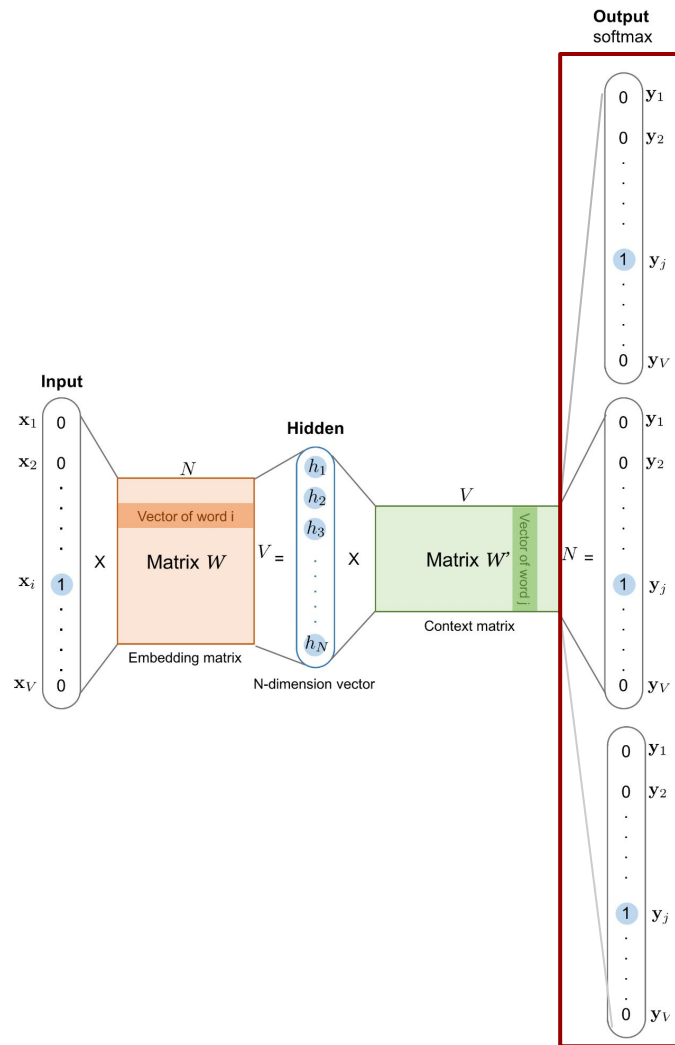


Skip-Gram

The output are m N -dimension vectors for the output representing all the predicted words in the context

V is the size of the vocabulary

These vectors are computed by multiplying the hidden vector by the matrix W' and applying the **Softmax Activation Function**

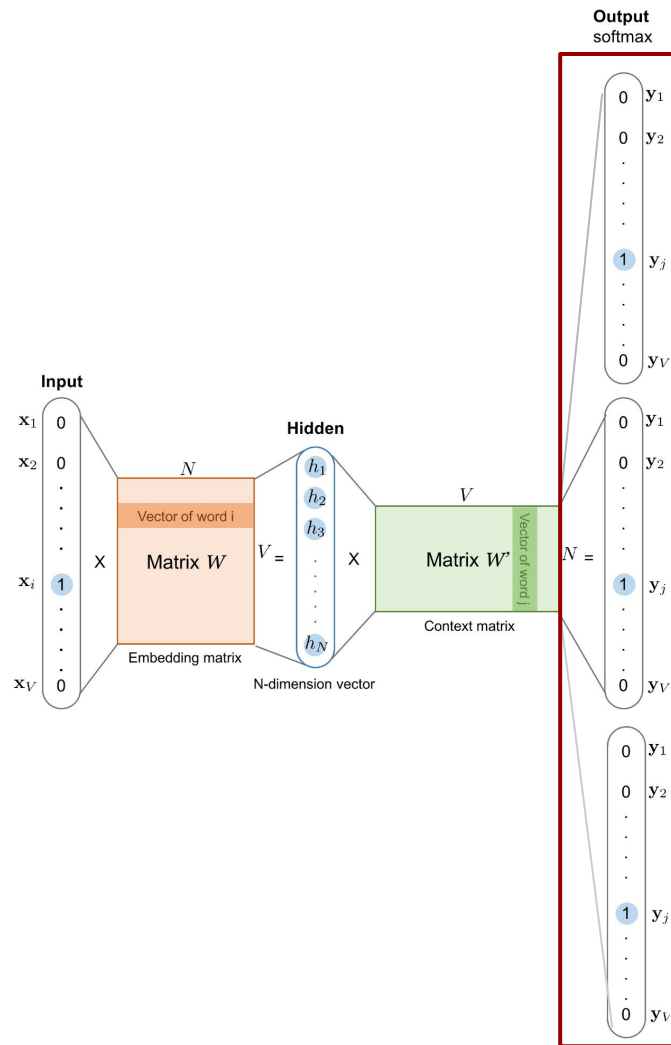


Skip-Gram

$$u[k] = \sum_{j=1}^N h[j] W'[j, k]$$

Softmax Activation Function

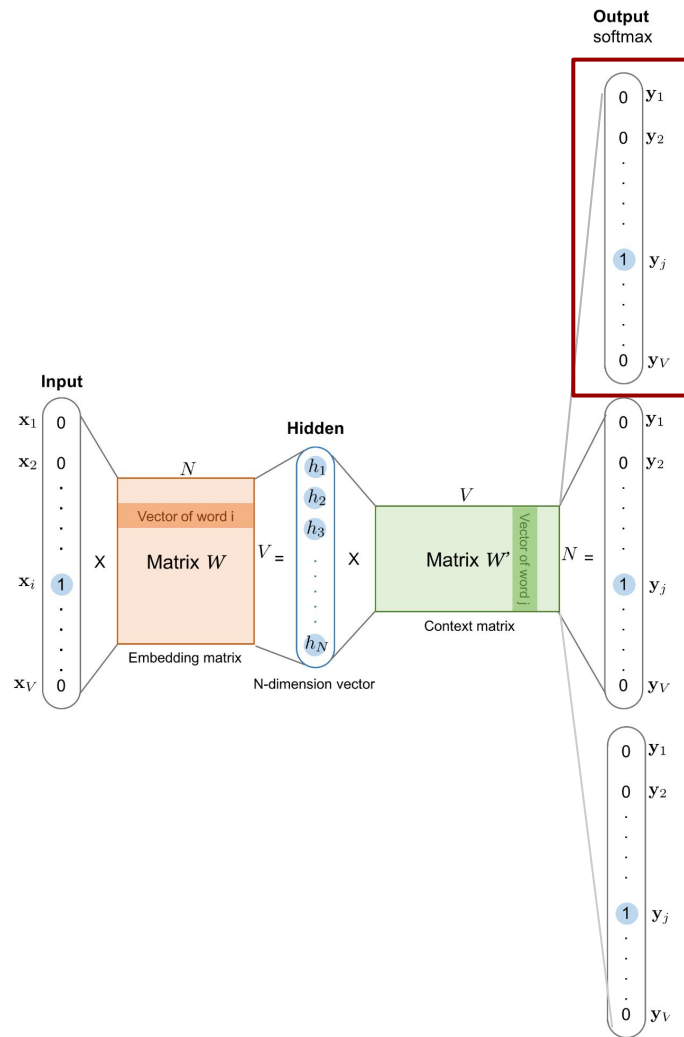
$$y_m[k] = \mu(u[k]) = \frac{e^{u[k]}}{\sum_{l=1}^V e^{u[l]}}$$



Skip-Gram

Ideally, we would like to have an output vectors like **one-hot encoded vector** ($V-1$ zeros, only one 1)

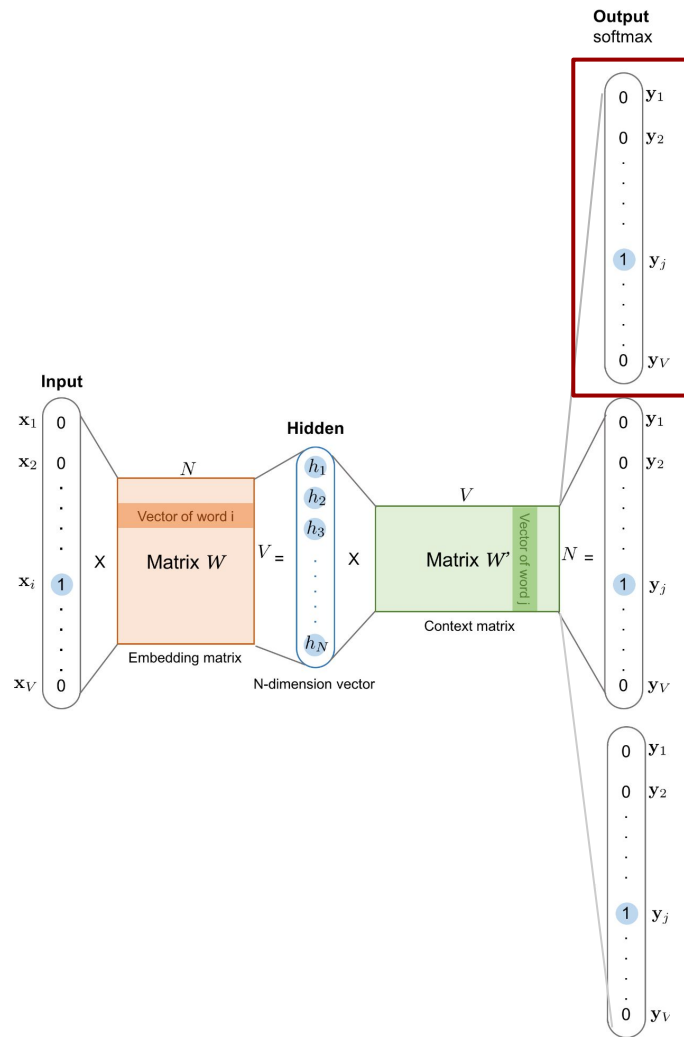
In reality, each element is a real value belonging to $[0, 1]$ representing the **conditional probability** for each word of being the context word.



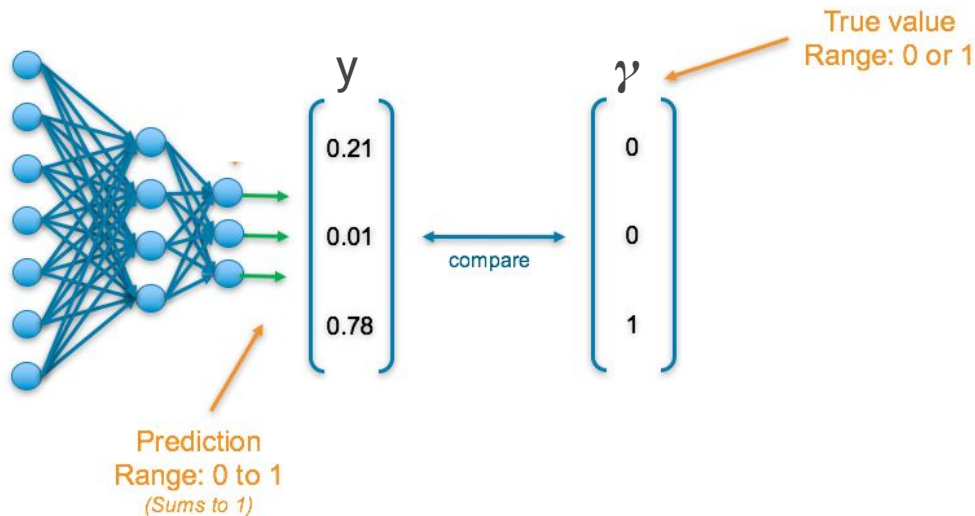
Skip-Gram

The sum of all elements belonging to each predicted vector must be **equal to 1**

The target word is selected as the word with the **highest probability** in y .



Skip-Gram: Error

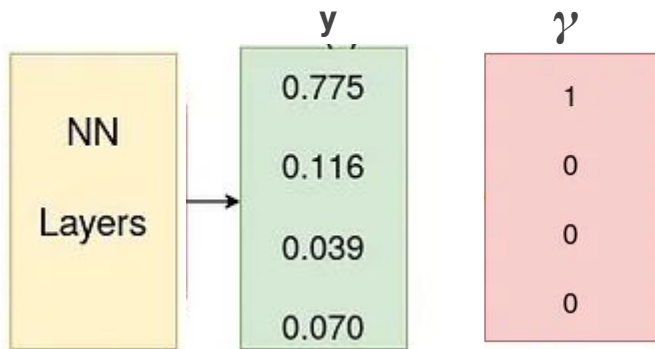


The prediction is not always the same as the the actual value.

The distance between the predicted value and the actual value is the **error**.

We would like to have a minimum error. So we need to define a **Loss Function**

Skip-Gram: Cross Entropy

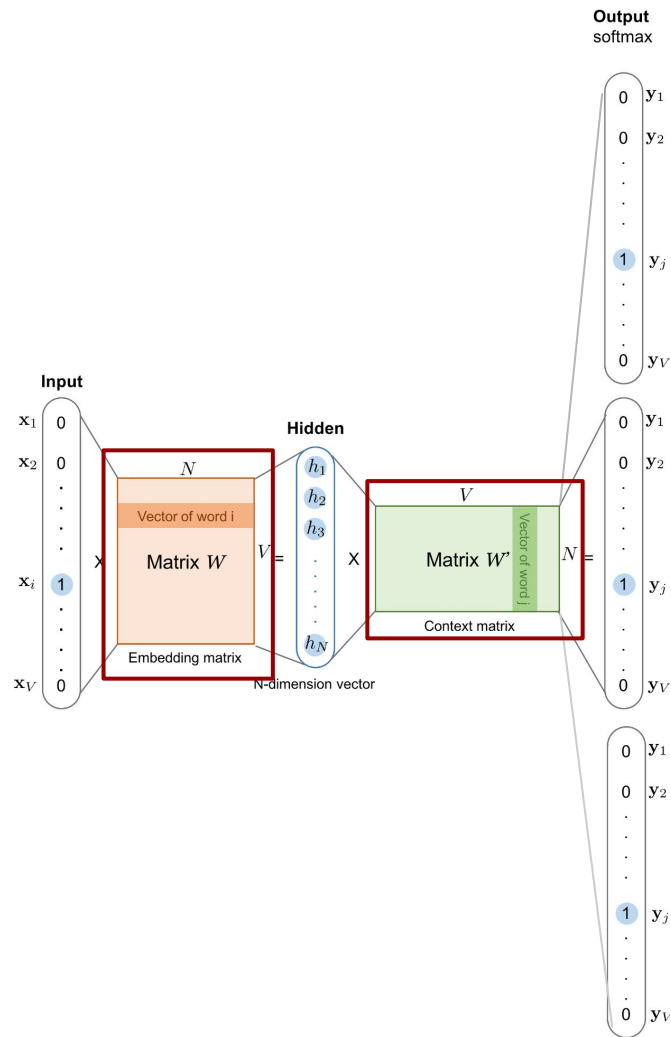


Both y and γ can be considered as **distributions**, so we can use the **Cross Entropy**.

$$L(y, \gamma) = - \sum_{m=1}^M \sum_{k=1}^V \gamma[k] \log_e(y_m[k])$$

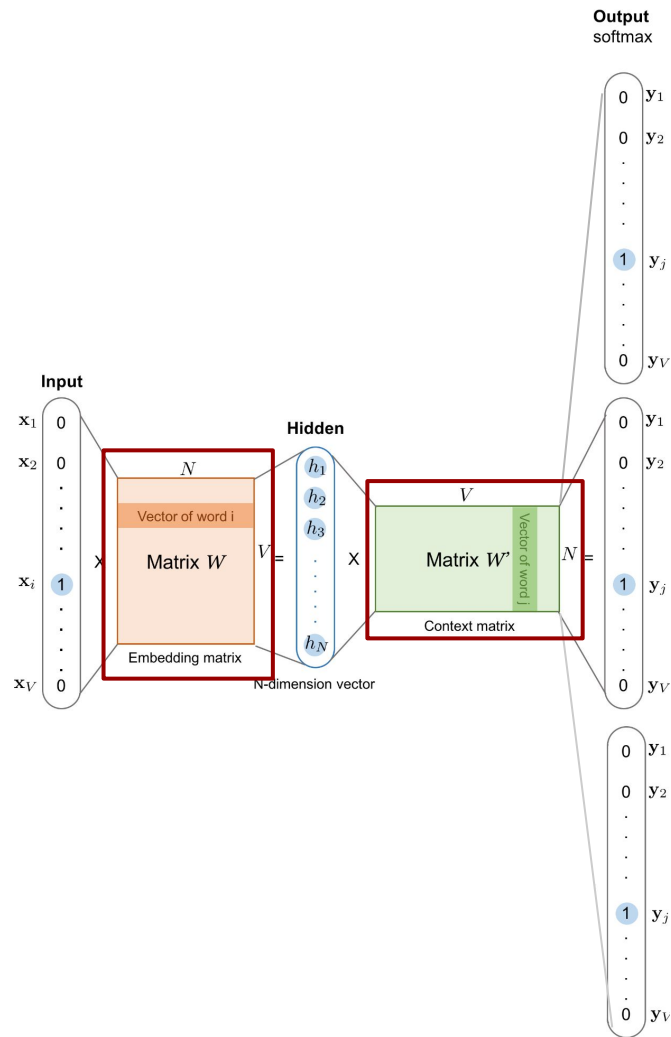
Skip-Gram

To reduce the error between the predicted context words and the real context words, Matrices W and W' are optimized through forward and **backward** propagations.



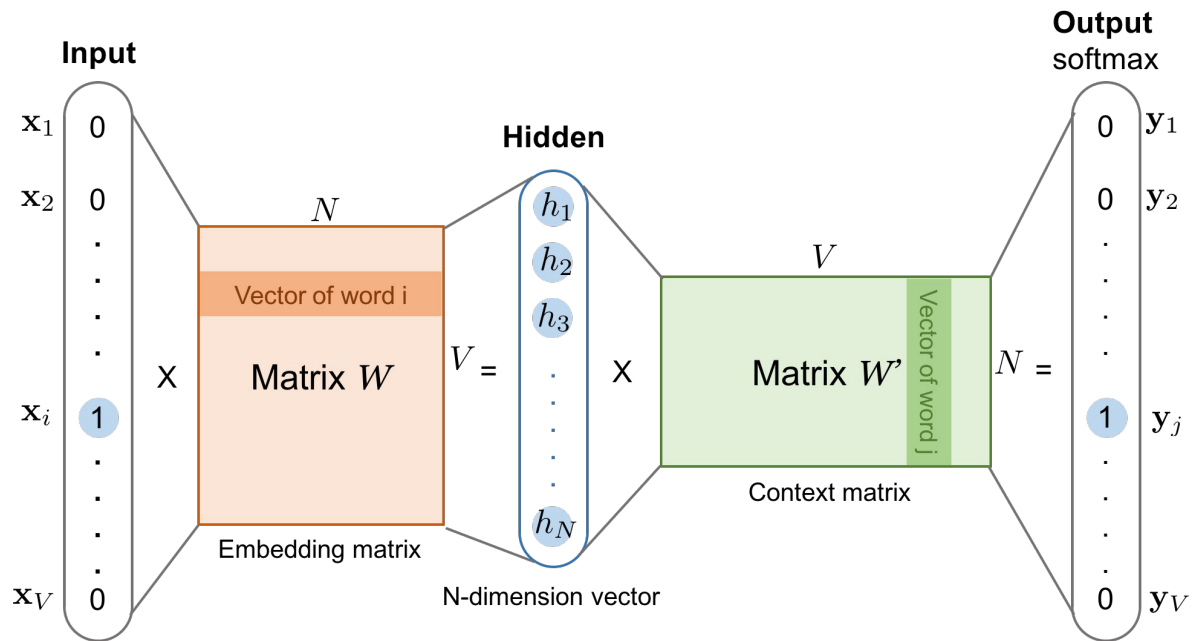
Skip-Gram

Parameters are adjusted running the Backpropagation phase and the **Stochastic Gradient Descent**



Vectorization

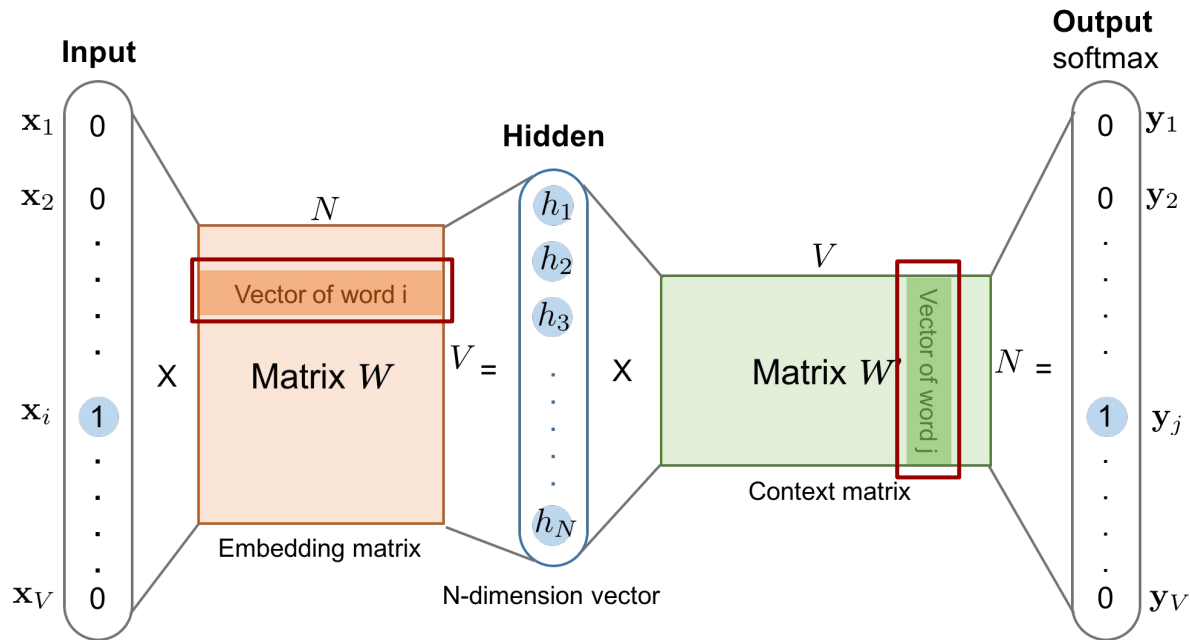
How vectors are really computed?



Supposing to have only one word in the surrounding context, the shape of the neural network will be like this.

This NN tries to predict the following word given a center word

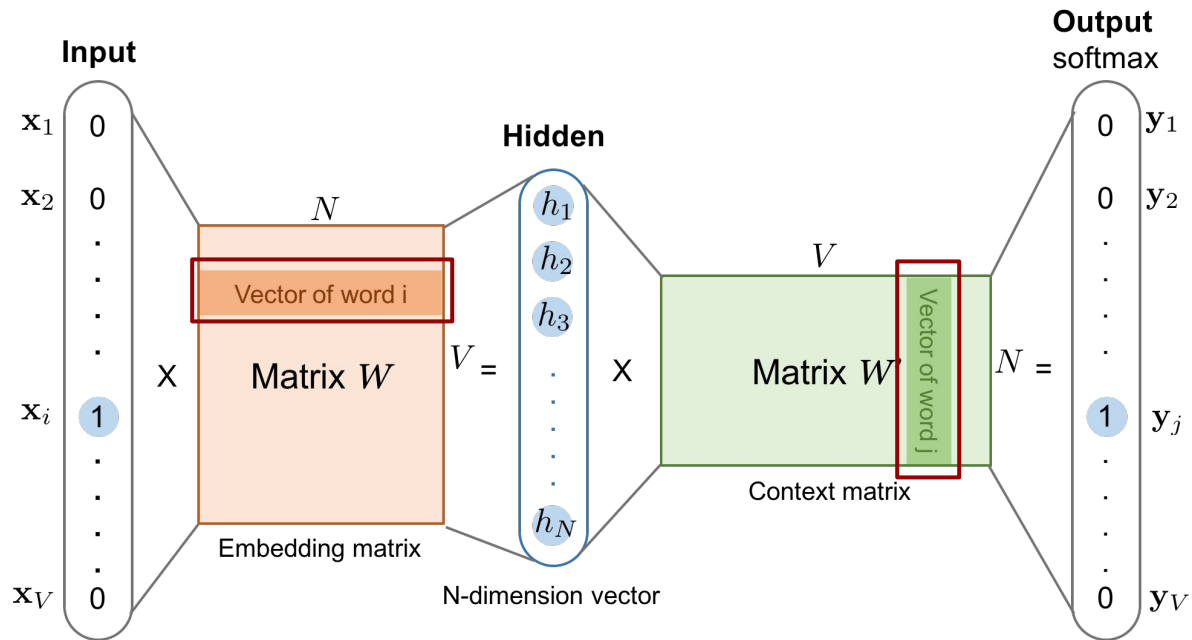
How vectors are really computed?



A row in the matrix W represents the **word emembedding** for a word in the vocabulary

A column in matrix W' encodes the **meaning of words as context**

How vectors are really computed?



Despite the name, W' is independent of W , not a transpose or inverse or whatsoever.

N is the size of the embedded word

Word2Vec Tool

Download and compile Word2Vec

Open <https://code.google.com/archive/p/word2vec/>

Select Source > Download > source-archive.zip

```
unzip source-archive.zip
```

```
cd word2vec/trunk
```

```
make
```

Download the corpus text

wget <http://mattmahoney.net/dc/text8.zip>

unzip text8.zip

text8 is a text corpus containing 16,718,843 words

text8 contains 71,291 unique words (to be inserted into the vocabulary)

Word2Vec

The word2vec tool takes a text corpus as input and produces the word vectors as output. It first constructs a **vocabulary** from the training text data and then learns **vector representation** of words. The resulting word vector file can be used as features in many natural language processing and machine learning applications.

Run Word2Vec

```
./word2vec
-train text8          // text corpus
-output vectors.bin   // word vector file
-cbow 1              // 1 = Continuous Bag-of-Words (CBOW) 0 = Skip-gram model (SG)
-size 200             // vector dimensionality
-window 8            // size of the context window
-threads 20          // number of threads to use
-binary 1            // the format of the output word vector file (0 = text, 1 = binary)
-iter 15             // Number of iterations (epochs) over the corpus
```

Running Similarities

Suppose we want to compute similarities between the word *learning* and all the words in the dataset:

```
$ ./distance vectors.bin learning
```

Word	Cosine distance
educational	0.539611
learners	0.520408
teaching	0.514668
learn	0.490442
knowledge	0.487726
cognitive	0.477808
education	0.476150

Running Linguistic Regularities

paris - france + berlin = germany

```
$ ./word-analogy vectors.bin Paris France Berlin
```

germany	0.630140
reunified	0.590625
gdr	0.503846
austria	0.486030
russia	0.482404
rhineland	0.477394
germanies	0.461999
dortmund	0.444944
reunification	0.439780

Where to obtain training data

The quality of the word vectors increases significantly with amount of the training data. For research purposes, you can consider using data sets that are available on-line:

- First Billion Characters from wikipedia: <http://mattmahoney.net/dc/enwik9.zip>
- Latest Wikipedia dump:
<http://dumps.wikimedia.org/enwiki/latest/enwiki-latest-pages-articles.xml.bz2>
- ...

Pretrained Word and Phrase Vectors

Google News: [GoogleNews-vectors-negative300.bin.gz](#) (1.5GB)

The dataset contains vectors trained on part of Google News dataset (about 100 billion words). The model contains 300-dimensional vectors for 3 million words and phrases.

Pretrained Italian Word Vectors

[Italian Word Embeddings | Italian Natural Language Processing Lab](#)

two dataset:

1. itWaC: billion word corpus constructed from the Web limiting the crawl to the .it domain and using medium-frequency words from the Repubblica corpus and basic Italian vocabulary lists as seeds
2. Twitter: 46.935.207 tweets

Document Embedding

Doc2Vec

Tool for transforming a document into a vector

A vector represents the theme or the overall meaning of a document

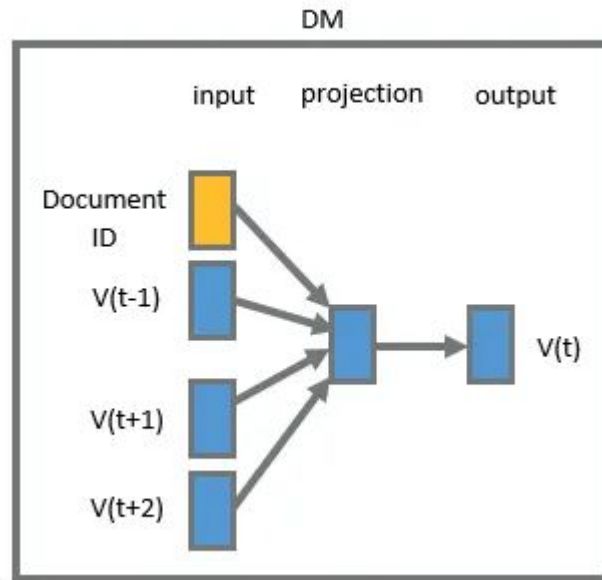
Two training model:

- Distributed Memory Model Of Paragraph Vectors (PV-DM), which is similar to the Continuous Bag of Words Model (CBOW)
- Paragraph Vector With A Distributed Bag Of Words (PV-DBOW), which is similar to the Skip-Gram Model

Distributed Memory Model

Similar to CBOW model

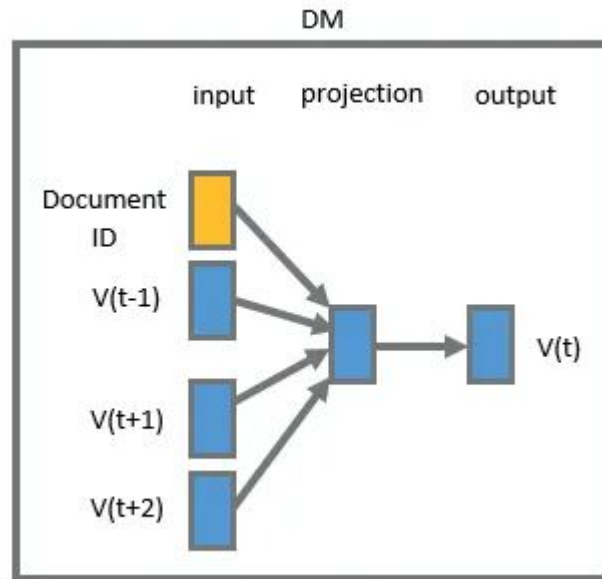
It tries to predict target word given its surrounding context words with the **addition of a Document ID**



Distributed Memory Model

The context words are used to predict a target word, and the document ID is used to capture the **overall meaning** of the document.

The projection layer is responsible for creating the **word vectors** and **document vectors**



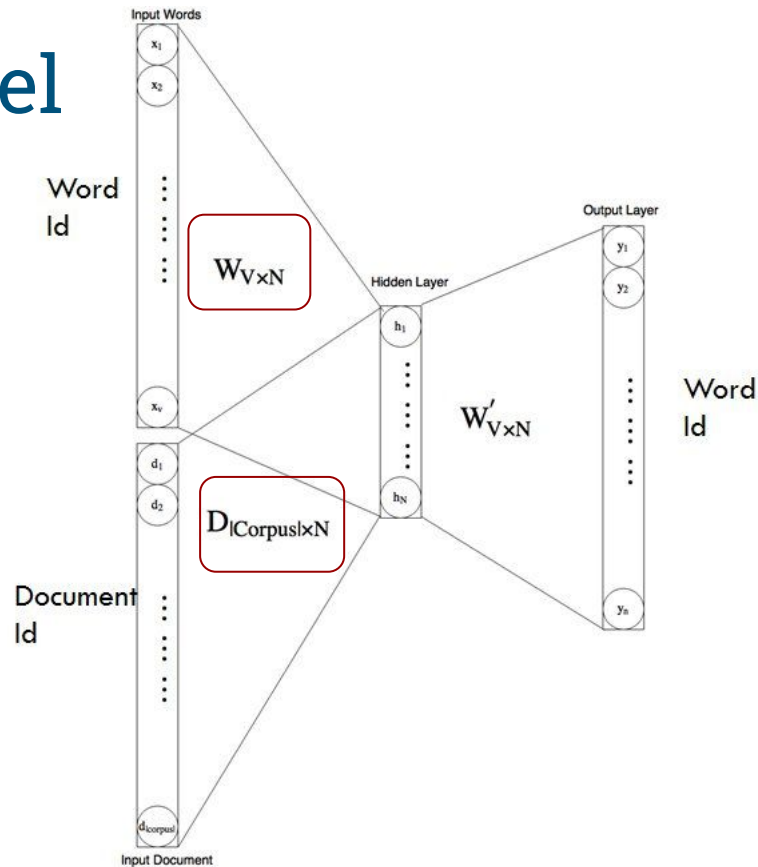
Distributed Memory Model

Two matrices:

W: $V \times N$ matrix for word embedding

D: $C \times N$ matrix for document embedding

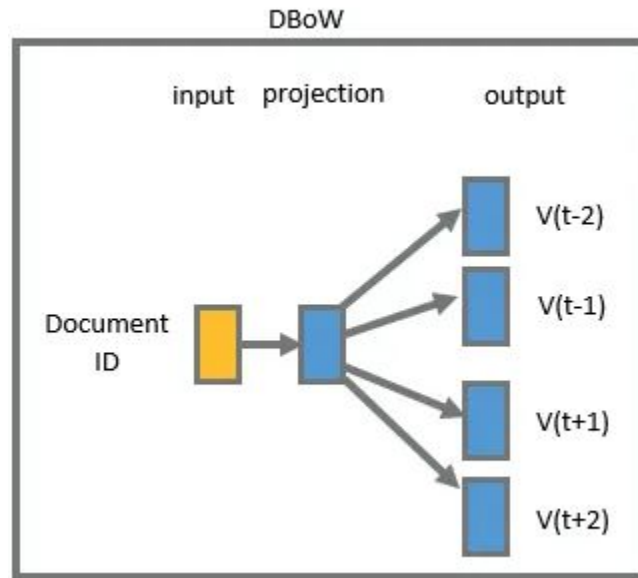
C is the number of documents in the collection



Distributed Bag-Of-Words Model

Similar to Skip-Gram model

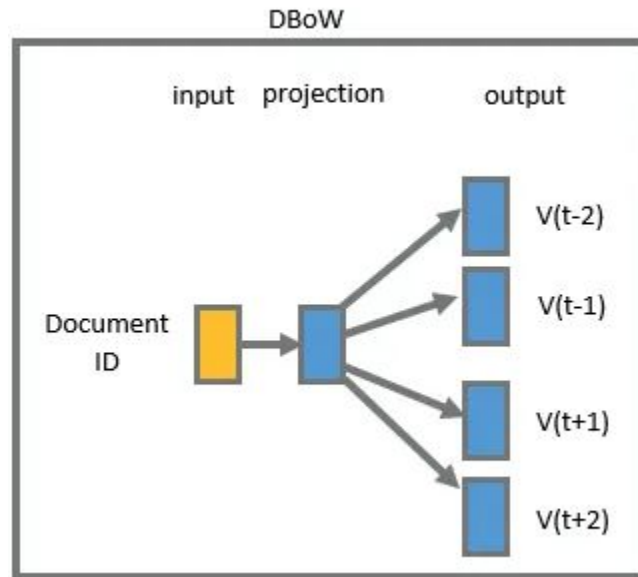
Instead of using the target word as the input, It takes the document ID as the input and tries to **predict the probability of each word** in the document given only the document vector



Distributed Bag-Of-Words Model

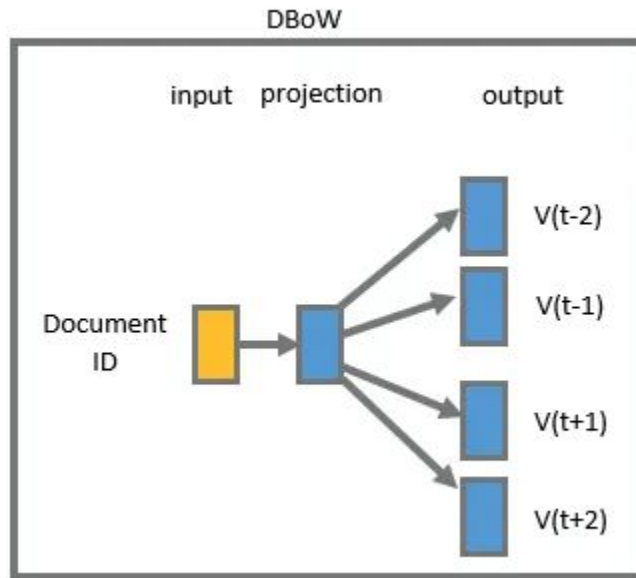
Similar to Skip-Gram model

It focuses on understanding how **words are distributed** in a text, rather than their meaning



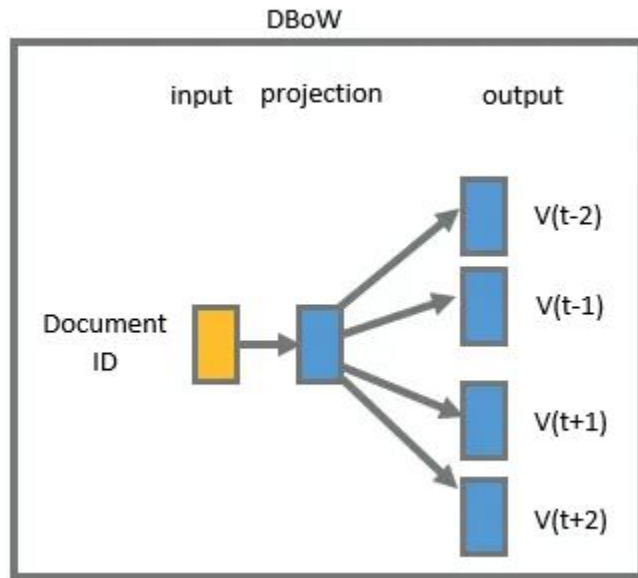
Distributed Bag-Of-Words Model

The model does not take into account the order of the words in the document, treating the document as a collection or “bag ” of words.



Distributed Bag-Of-Words Model

This makes the DBOW architecture faster to train than DM, but potentially **less powerful in capturing the meaning** of the documents



FastText



FastText

It is an extension of the popular Word2Vec model.

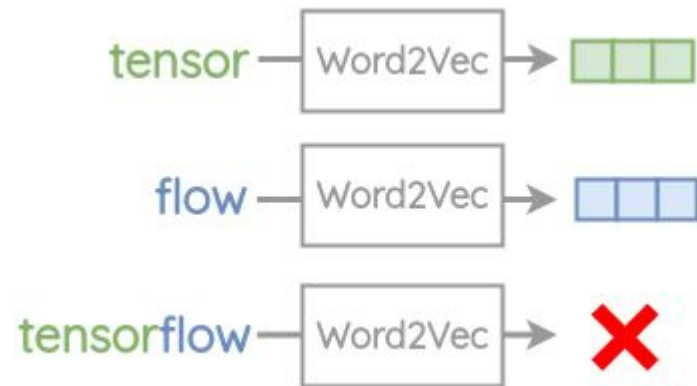
It has been developed by Facebook in 2017

Tools, libraries and already-available vector representation for 157 languages are available of **<https://fasttext.cc/>**

FastText: limits #1 of Word2Vec

In Word2Vec, an embedding is created for each word.

For missing words, there is no way to get the vector representation, not even if the missing word is made by the concatenation of two available words



FastText: limits #2 of Word2Vec

In Word2Vec, each word is learned uniquely **based on the context** it appears in.

Vectors are created without utilizing the **internal structure** of the word to make the process more efficient.

Shared radical

eat eats eaten eater eating

FastText: how it works

The idea behind FastText: instead of creating embedding of just words, it creates embeddings of subword (called **n-grams**).

n is an integer ranging between 3 and 6

To capture meaning for suffixes/prefixes, **angular brackets** are added at the beginning and at the end of each word.

Embeddings can be created using either CBoW or SkipGram.

FastText: how it works

eating → <eating>

We add the brackets

<ea, eat, ati, tin, ing, ng>

n = 3 - 6 three-grams

<eat, eati, atin, ting, ing>

n = 4 - 5 four-grams

<eati, eatin, ating, ting>

n = 5 - 4 five-grams

<eatin, eating, ating>

n = 6 - 3 six-grams

FastText: how it works

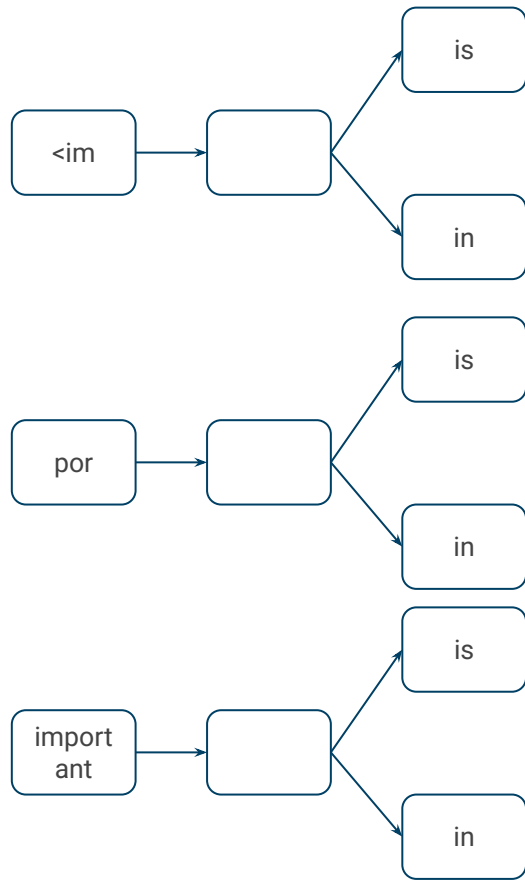
- During training, FastText learns embeddings **for each n-gram** and, if the word itself is common enough, it also learns an embedding for the **whole word**
 - FastText considers a word as common if it appears at least **5 times** into the corpus
- Embedding can make use of CBoW or SkipGram
- Each n-gram and the whole word is considered as **center word**
- The context instead is the **context** of the whole word

FastText: how it works

For example, the sentence is

strategy is important in playing chess

- center word: important
- n-grams: <im, imp, mpo, por, ...
- size of window: 1
- model: skip-gram



FastText: how it works

After completing the learning process, when a word vector is needed, FastText computes the word vector dynamically by **averaging the embeddings of all relevant n-grams** and the whole-word vector if it exists

$$\text{vector}_{\text{word}} = \frac{1}{N+1} \left(\sum_{i=1}^N \text{vector}_{\text{n-gram}_i} + \text{vector}_{\text{whole-word}} \right)$$

N is the total number of n-grams (3-grams, 4-grams, 5-grams, and 6-grams) for the word